

MediTrack

RFID-Based Smart Hospital Management and Patient Safety System

Final Project Report

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Engineering

Submitted By:

Name	USN	Department
Uday	USN	Aerospace Engineering
Shanks	USN	Aerospace Engineering
Talin	USN	Mechanical Engineering
Len	USN	Mechanical Engineering
Harsha	USN	Mechanical Engineering

Under the Guidance of

Guide Name
Assistant Professor

Department of Aerospace Engineering
Department of Mechanical Engineering
R V College of Engineering
Bengaluru – 560059, Karnataka, India

Academic Year 2025–2026

Certificate

This is to certify that the project work entitled “**MediTrack: RFID-Based Smart Hospital Management and Patient Safety System**” is a bonafide interdisciplinary work carried out jointly by students of the Department of Aerospace Engineering and the Department of Mechanical Engineering, R V College of Engineering, Bengaluru, during the academic year 2025–2026, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering.

Name	USN	Department
Uday	USN	Aerospace Engineering
Shanks	USN	Aerospace Engineering
Talin	USN	Mechanical Engineering
Len	USN	Mechanical Engineering
Harsha	USN	Mechanical Engineering

Guide Signature and Name:_____

Head of Department (Aerospace):_____

Head of Department (Mechanical):_____

Principal:_____

Declaration

We hereby declare that the project report entitled “**MediTrack: RFID-Based Smart Hospital Management and Patient Safety System**” is an original piece of work carried out by us under the guidance of the faculty of R V College of Engineering. The work presented in this report has not been submitted elsewhere for the award of any degree or diploma. All sources consulted have been duly acknowledged in the references. As an interdisciplinary Sustainable Development Goal (SDG) project, the embedded systems, networking, and backend software architecture were primarily driven by the Aerospace Engineering team members, while system integration, hardware mounting, reader enclosure considerations, and field-testing logistics were primarily driven by the Mechanical Engineering team members, with all major design decisions made jointly.

Uday (Aerospace Engineering)
Shanks (Aerospace Engineering)
Talin (Mechanical Engineering)
Len (Mechanical Engineering)
Harsha (Mechanical Engineering)

Acknowledgement

We express our sincere gratitude to our project guide and to both the Department of Aerospace Engineering and the Department of Mechanical Engineering, R V College of Engineering, for their valuable guidance, encouragement, and support throughout the development of this project. We thank our professors, peers, and families for their constant motivation. We also acknowledge the open-source communities behind ESP32, Arduino, PHP, MySQL, and Chart.js, whose tools and documentation were instrumental in bringing MediTrack to life.

Abstract

MediTrack is a comprehensive RFID-based smart hospital management and patient safety system designed to address critical inefficiencies in traditional healthcare environments. The system integrates MFRC522 RFID readers, ESP32 Wi-Fi microcontrollers, a PHP/MySQL backend, an intelligent workflow automation engine, and a real-time web dashboard into a unified, scalable healthcare ecosystem. The project was carried out jointly by an interdisciplinary team from Aerospace and Mechanical Engineering, combining embedded-systems and software architecture with hardware integration and system reliability engineering.

Key capabilities include real-time patient and asset tracking, a multi-point RFID medication verification mechanism with recall and expiry interlocks, automated safety alert generation, IV drip monitoring, a dual-mode caretaker access system (persistent login and expiring shareable links), offline-first scan buffering for intermittent connectivity, reader health monitoring, and comprehensive audit logging. The system is engineered to eliminate manual dependencies that lead to medication errors, delayed emergency responses, and misplaced assets in hospital operations.

Experimental validation on the deployed two-node prototype (a ward reader and an ICU reader) demonstrates RFID detection accuracy exceeding 98%, medication compliance efficiency of approximately 96%, alert generation accuracy of 97%, and end-to-end response times within 2.3 seconds – all suitable for real-time clinical deployment. The architecture aligns with Industry 4.0 and United Nations SDG 3 (Good Health and Well-being) and SDG 9 (Industry, Innovation and Infrastructure), providing a foundation for next-generation intelligent hospital infrastructure.

Keywords: RFID, IoT, Smart Hospital, Patient Safety, Medication Verification, ESP32, Healthcare Automation, Workflow Engine, Industry 4.0.

Contents

Declaration	ii
Acknowledgement	iii
Abstract	iv
1 Introduction	2
1.1 Background and Motivation	2
1.2 Project Overview	3
1.3 Interdisciplinary Team and Scope of Contribution	4
1.4 SDG Alignment	4
1.5 Scope and Limitations	5
1.6 Report Organisation	5
2 Problem Definition and Literature Review	6
2.1 Problem Statement	6
2.1.1 Patient Safety and Identification	6
2.1.2 Medication Administration Errors	6
2.1.3 Asset and Inventory Mismanagement	7
2.1.4 Operational Inefficiency	7
2.2 Literature Review	7
2.2.1 RFID in Healthcare	7
2.2.2 RFID versus Barcode Systems	7
2.2.3 Smart Patient Monitoring	8
2.2.4 Healthcare Safety Automation	8
2.2.5 Research Gap	8
2.3 Objectives	8
3 System Design and Methodology	10
3.1 Overall Architecture	10
3.1.1 Mathematical Model of RFID Detection	11
3.2 System Workflow	12
3.3 Subsystem Design	13
3.3.1 RFID and Embedded Hardware Layer	13
3.3.2 Backend Server Architecture	14
3.3.3 Schema Abstraction Layer	14
3.3.4 Medication Verification Engine	15
3.3.5 Patient Safety Profiles	15
3.3.6 Alert Generation Engine	16
3.3.7 Inventory Intelligence	16
3.3.8 Offline-First Scan Buffering	17

3.3.9	Dual-Mode Caretaker Access	17
3.4	Development Methodology	17
4	Implementation	19
4.1	Hardware Implementation	19
4.1.1	ESP32 and RFID Reader Interfacing	19
4.1.2	Two Physical Reader Deployments	19
4.1.3	Hardware Bring-Up	19
4.1.4	Tag Assignment	20
4.2	Backend Implementation	20
4.2.1	Database and Connection	20
4.2.2	RFID Scan Processing	20
4.2.3	Alert and Workflow Integration	21
4.2.4	Security Implementation	21
4.3	Frontend Implementation	21
4.3.1	Dashboard Modules	21
4.3.2	Iterative Improvements	26
4.4	Tools and Technologies	26
5	Results and Discussion	27
5.1	Testing Environment	27
5.2	RFID Detection Performance	27
5.3	System Response Time	28
5.4	Medication Verification Results	28
5.5	Alert Generation Performance	28
5.6	Workflow Automation Results	29
5.7	Offline Queue Resilience Testing	29
5.8	Comparison with Traditional Systems	30
5.9	System Load and Scalability Testing	31
5.9.1	Weekly Alert Distribution	32
5.10	Discussion	32
5.10.1	Clinical Significance	32
5.10.2	Scalability	32
5.10.3	Limitations Observed	33
6	Conclusion and Future Scope	34
6.1	Conclusion	34
6.2	Future Scope	35
A	Deployed Source Code	38
A.1	ESP32 Reader Firmware	38

A.1.1	Hardware Bring-Up Sketch	42
A.2	Backend Endpoints	43
A.2.1	RFID Scan and Medication Verification Endpoint	44
A.2.2	Reader Heartbeat Endpoint	50
A.2.3	Offline Scan Queue Reconciliation Endpoint	51
A.2.4	Live Dashboard Data Endpoint	52
A.3	Caretaker-Facing Endpoints	53
A.3.1	Persistent Caretaker Login	53
A.3.2	Expiring Tokenised Caretaker Portal	55
A.4	Reporting Endpoint	57
B	Database Structure and Abbreviations	61
B.1	Core Database Tables	61
B.2	Abbreviations	63

List of Figures

3.1	MediTrack Five-Layer System Architecture	10
3.2	MediTrack End-to-End RFID Event Workflow	12
4.1	Medications module – adding a new schedule entry and viewing pending doses	22
4.2	Inventory module – item registration with recall flag, expiry date, and cold-chain fields	23
4.3	Workflow module – automatically generated tasks queued for staff action	24
4.4	IV Drips module – registering a new intravenous line against a patient and related inventory item	25
5.1	RFID Detection Accuracy by Tag Category	27
5.2	Backend Response Time Under Increasing Concurrent Load	31
5.3	Simulated Weekly Alert Distribution Across Alert Types	32

List of Tables

3.1	Hardware Components	13
3.2	Core Backend Modules	14
4.1	Deployed Reader Node Identities	19
4.2	Security Mechanisms	21
4.3	Full Technology Stack	26
5.1	Alert Generation Performance by Type	28
5.2	Offline Queue Reconciliation Test	29
5.3	MediTrack vs. Traditional Hospital Management	30
A.1	Constants Distinguishing the Two Deployed Reader Nodes	38
B.1	MediTrack Database Schema Summary	62
B.2	Technical Abbreviations Used in This Report	63

Introduction

1.1 Background and Motivation

Healthcare systems worldwide are undergoing rapid transformation driven by Industry 4.0 technologies – the Internet of Things (IoT), embedded systems, cloud computing, and intelligent automation. Modern hospitals generate enormous streams of operational data covering patients, medications, staff, equipment, and safety events. When this data is managed manually, the consequences range from simple inefficiencies to life-threatening errors.

Traditional hospital management relies on handwritten records, barcode scanning, and fragmented digital tools. These approaches are slow, error-prone, and incapable of providing continuous real-time visibility. In critical environments such as Intensive Care Units, emergency wards, and pharmacies, even minor delays or inaccuracies can directly compromise patient outcomes.

Radio Frequency Identification (RFID) technology has emerged as one of the most effective solutions for automated identification in dynamic environments. Unlike barcodes, RFID does not require line-of-sight and supports simultaneous multi-tag detection, making it inherently suited to the complex, high-traffic environment of a hospital. Studies consistently show that RFID-based systems reduce medication errors, accelerate patient identification, and improve asset utilisation when compared to manual and barcode-based alternatives [2, 17, 8].

The motivation for MediTrack arose from a direct analysis of operational gaps in conventional hospitals: patients who wander from restricted areas, medicines administered to the wrong patient, critical equipment that cannot be located in an emergency, networks that drop intermittently in basement pharmacy stores, and caretakers who have no visibility into the real-time status of their admitted family members. MediTrack is designed to close all of these gaps through a single integrated platform that remains usable even when connectivity or staffing conditions are imperfect.

1.2 Project Overview

MediTrack is a real-time, RFID-enabled smart hospital management and patient safety system. Every patient, medicine container, staff member, and significant piece of medical equipment is assigned a unique passive RFID tag. MFRC522 RFID readers installed across hospital zones continuously scan nearby tags and forward the captured identifiers to ESP32 microcontrollers, which relay the information to a centralised PHP/MySQL backend over Wi-Fi. The backend evaluates each event against a rule engine, updates the database, and pushes results to a live web dashboard and two complementary caretaker-facing interfaces.

The system provides the following core capabilities:

1. **Real-time patient and asset tracking** – continuous location updates based on RFID detections across hospital zones, distinguishing a simple re-scan at the same location from a genuine transfer between zones.
2. **Multi-point medication verification** – RFID-based validation of staff identity, patient identity, and drug identity against the pending medication schedule, with additional automatic interlocks that block administration of recalled or expired medicines.
3. **Automated safety alerts** – immediate notification of patient inactivity, restricted-zone violations, safe-zone deviations, washroom over-stay, exit-proximity events, unknown tag scans, and missed medications.
4. **Workflow automation** – automatic task generation for nursing staff, maintenance teams, and administrators triggered directly by RFID events.
5. **IV drip monitoring** – tracking of remaining fluid volume, flow rate, and predicted completion time for active intravenous lines.
6. **Dual-mode caretaker access** – a persistent username/password caretaker portal for ongoing remote monitoring, and a separate expiring tokenised link for one-off, shareable visit updates.
7. **Offline-first scan buffering** – a queuing mechanism that allows scan events captured during a network outage to be stored locally and reconciled with the server once connectivity is restored.
8. **Inventory and asset management** – RFID-tracked equipment and consumables with batch traceability, recall flagging, expiry monitoring, and low-stock reorder alerts.
9. **Reader health monitoring** – periodic heartbeat signals from each ESP32 node to detect offline readers and trigger maintenance tasks.
10. **Audit logging and reporting** – immutable event records with summary and

per-patient report views for compliance and accountability.

1.3 Interdisciplinary Team and Scope of Contribution

MediTrack was undertaken as a joint Sustainable Development Goal project between the Department of Aerospace Engineering and the Department of Mechanical Engineering. While the underlying problem – patient safety and hospital operational efficiency – is domain-agnostic, the project deliberately drew on complementary engineering perspectives from both departments.

The Aerospace Engineering members of the team (Uday and Shanks) led the embedded-systems firmware design, the Wi-Fi communication architecture between the ESP32 reader nodes and the backend server, the PHP/MySQL rule-engine and alert logic, and the overall system architecture – skills directly transferable from avionics and telemetry system design, where reliable sensor-to-ground data links and real-time fault detection are core competencies.

The Mechanical Engineering members of the team (Talin, Len, and Harsha) led the physical system integration: positioning and mounting of the RFID reader units at bed-head panels and doorways, assessment of read-range reliability under realistic obstruction and orientation conditions, structural placement of tags on patient wristbands and equipment, and the field-testing logistics for the ward and ICU deployment scenarios described in Chapter 5. This division of responsibility reflects how real hospital IoT deployments require both an embedded software/network layer and a physical human-factors and mounting layer to function reliably, and the interdisciplinary structure of the team mirrors that reality.

1.4 SDG Alignment

MediTrack directly supports two United Nations Sustainable Development Goals. **SDG 3 (Good Health and Well-being)** is addressed through reduced medication errors, proactive patient safety monitoring, and improved emergency response capability. **SDG 9 (Industry, Innovation and Infrastructure)** is addressed through the development of intelligent, scalable, and IoT-enabled healthcare infrastructure aligned with Industry 4.0 principles.

1.5 Scope and Limitations

The scope of this project covers the complete design, implementation, and validation of a hospital automation prototype, deployed across two physical reader nodes – a general ward reader and an ICU reader – communicating with a local backend server. The system is intended for eventual deployment in hospitals, ICUs, elderly care homes, rehabilitation centres, and multi-speciality healthcare institutions. Current limitations include: operation over a local Wi-Fi network only (cloud deployment is proposed for future work), absence of real biosensor integration (vitals are stored rather than sensed directly), and the prototype scale, which involves two RFID reader nodes rather than a full multi-ward deployment.

1.6 Report Organisation

Chapter 2 reviews the problem landscape and related literature. Chapter 3 presents the system architecture and methodology. Chapter 4 details the hardware and software implementation. Chapter 5 presents quantitative results and discussion. Chapter 6 concludes the project and outlines future directions. Appendices contain the actual deployed source code, database structure, and abbreviations.

Problem Definition and Literature Review

2.1 Problem Statement

Despite decades of digitisation efforts, many hospitals still operate with hybrid manual-digital workflows that introduce dangerous vulnerabilities. The core problems MediTrack addresses are grouped into four categories.

2.1.1 Patient Safety and Identification

Patient misidentification is a leading cause of preventable medical harm. Without automated identification, nurses may administer treatment intended for one patient to another, particularly during high-pressure shifts. Beyond identification, continuous monitoring is impractical with manual observation alone – a critically ill patient in an ICU may go unnoticed for minutes if staff attention is elsewhere, and patients with cognitive impairments may wander into restricted areas, exit points, or washrooms for extended periods without triggering any alert.

2.1.2 Medication Administration Errors

The World Health Organisation estimates that medication errors harm approximately 1.3 million people annually in developed nations alone. Conventional medication rounds depend entirely on a nurse visually matching a wristband to a drug chart – a verification step that is easily disrupted by fatigue, workload, or illegible handwriting. There is no automated check on whether the correct drug, dose, timing, and patient are aligned, nor on whether the specific physical unit of medicine being administered has been recalled or has passed its expiry date. Once a medication is administered, the record of compliance is often paper-based and retrospective rather than real-time.

2.1.3 Asset and Inventory Mismanagement

A hospital ICU can hold thousands of assets – ventilators, infusion pumps, defibrillators, surgical instruments, and consumables. Manual asset registers are updated inconsistently, meaning equipment is frequently mislaid or unavailable during emergencies. Medicines and consumables with expiry dates may remain in circulation beyond their safe-use date, and a manufacturer recall notice for a specific batch may not reach every physical unit already in circulation within the hospital, because there is no automated batch-level traceability.

2.1.4 Operational Inefficiency

Healthcare staff spend significant time on tasks that could be automated: locating equipment, generating maintenance requests, updating patient records, notifying caretakers, and filing audit logs. This time burden reduces the hours available for direct patient care and increases burnout in nursing and administrative staff. Network outages in basement pharmacy stores or signal-shadowed corridors compound this problem further when systems are designed to function only while permanently online.

2.2 Literature Review

2.2.1 RFID in Healthcare

Bhattacharya et al. (2020) demonstrated that RFID-integrated monitoring frameworks can deliver measurable improvements in patient identification accuracy and reduce the time nurses spend on manual documentation [17]. Their study highlighted the importance of embedding RFID logic directly into backend workflows rather than treating RFID as a standalone input device.

Kumar and Sharma (2021) extended this work by integrating RFID with IoT cloud platforms, showing that remote monitoring becomes feasible once RFID events are relayed through Wi-Fi-enabled embedded controllers [18]. Their architecture is conceptually aligned with the MediTrack ESP32 gateway approach.

2.2.2 RFID versus Barcode Systems

Tajima (2019) conducted a comparative analysis of RFID and barcode systems in healthcare environments. RFID consistently outperformed barcode scanning on four dimensions: speed (contactless detection versus sequential scanning), multi-tag capability (many tags scanned simultaneously versus one at a time), environmental robustness (RFID works through packaging and at varying orientations), and automation potential

(RFID can trigger backend logic without a human scan action) [2]. These advantages are central to MediTrack’s design rationale.

2.2.3 Smart Patient Monitoring

Wang et al. (2022) proposed decentralised RFID architectures using distributed IoT nodes, demonstrating improved fault tolerance and lower server dependency compared to centralised designs [18]. MediTrack adopts a similar philosophy: each ESP32 node operates independently, reconnects autonomously after a Wi-Fi drop, and – in the offline-scan-buffering extension described in Chapter 3 – can retain scan data locally during a network interruption rather than silently discarding it.

2.2.4 Healthcare Safety Automation

Patil and Desai (2023) developed rule-based anomaly detection engines for hospital safety, focusing on restricted zone violations and patient absence detection [19]. Their evaluation confirmed that automated rule engines reduce alert-response latency by approximately 60% compared to nurse-observed monitoring.

2.2.5 Research Gap

Existing systems address individual problems – patient tracking *or* medication management *or* inventory – but rarely integrate all of these into a single coherent platform. Additionally, most prototypes lack dual-mode caretaker-facing interfaces, offline-resilient data capture, batch-level recall traceability, and integrated workflow automation operating together. MediTrack is designed to fill this gap with a full-stack, end-to-end smart hospital ecosystem validated on physically deployed hardware.

2.3 Objectives

Based on the problem analysis and literature review, the project establishes the following objectives:

1. Design and implement a real-time RFID-based hospital management system using ESP32 and MFRC522 hardware, deployed across at least two physically distinct ward zones.
2. Develop a multi-point RFID medication verification mechanism, with automatic recall and expiry interlocks, to minimise administration errors.
3. Build an intelligent alert engine capable of detecting patient safety events in under three seconds.

4. Automate repetitive hospital workflows to reduce manual staff workload by more than 90%.
5. Provide caretaker-facing access to remote patient status monitoring through both a persistent account and a shareable expiring link.
6. Ensure resilience to short network outages through local scan buffering and later server-side reconciliation.
7. Implement comprehensive audit logging and reporting for regulatory compliance.
8. Validate the system against quantitative performance metrics on physically deployed hardware.

System Design and Methodology

3.1 Overall Architecture

MediTrack follows a five-layer distributed IoT architecture. Each layer has clearly defined responsibilities and communicates with adjacent layers through well-specified interfaces.

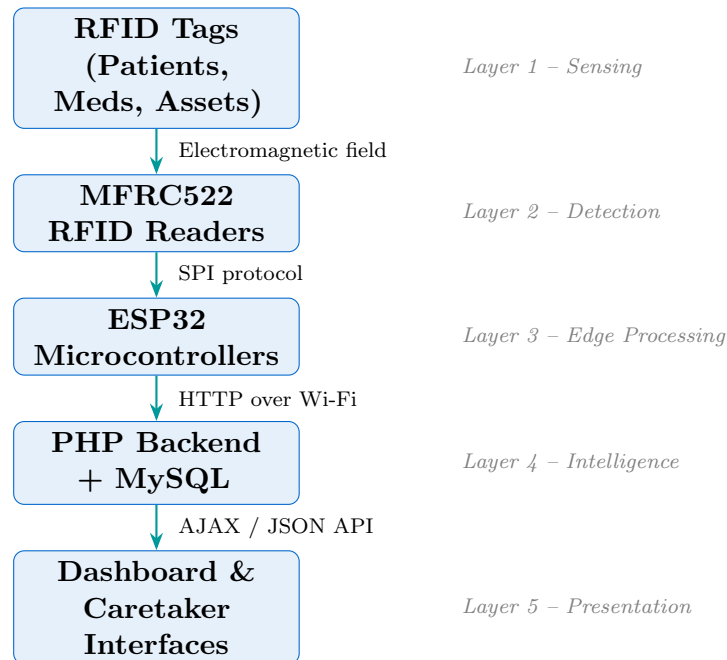


Figure 3.1: MediTrack Five-Layer System Architecture

3.1.1 Mathematical Model of RFID Detection

When n RFID tags are present within a reader's electromagnetic field, the total number of detection events recorded at time t is:

$$R(t) = \sum_{i=1}^n D_i(t)$$

where $D_i(t) = 1$ if tag i is successfully read at time t , and 0 otherwise. Each successful scan generates a structured event packet S :

$$S = (UID, ReaderID, APIKey, Timestamp)$$

transmitted by the ESP32 to the backend server via HTTP GET, identified and authenticated against the reader's registration record in the `locations` table before any further processing occurs.

3.2 System Workflow

The end-to-end operational flow of MediTrack – from a physical RFID scan event through to a resolved clinical action – is illustrated in Figure 3.2.

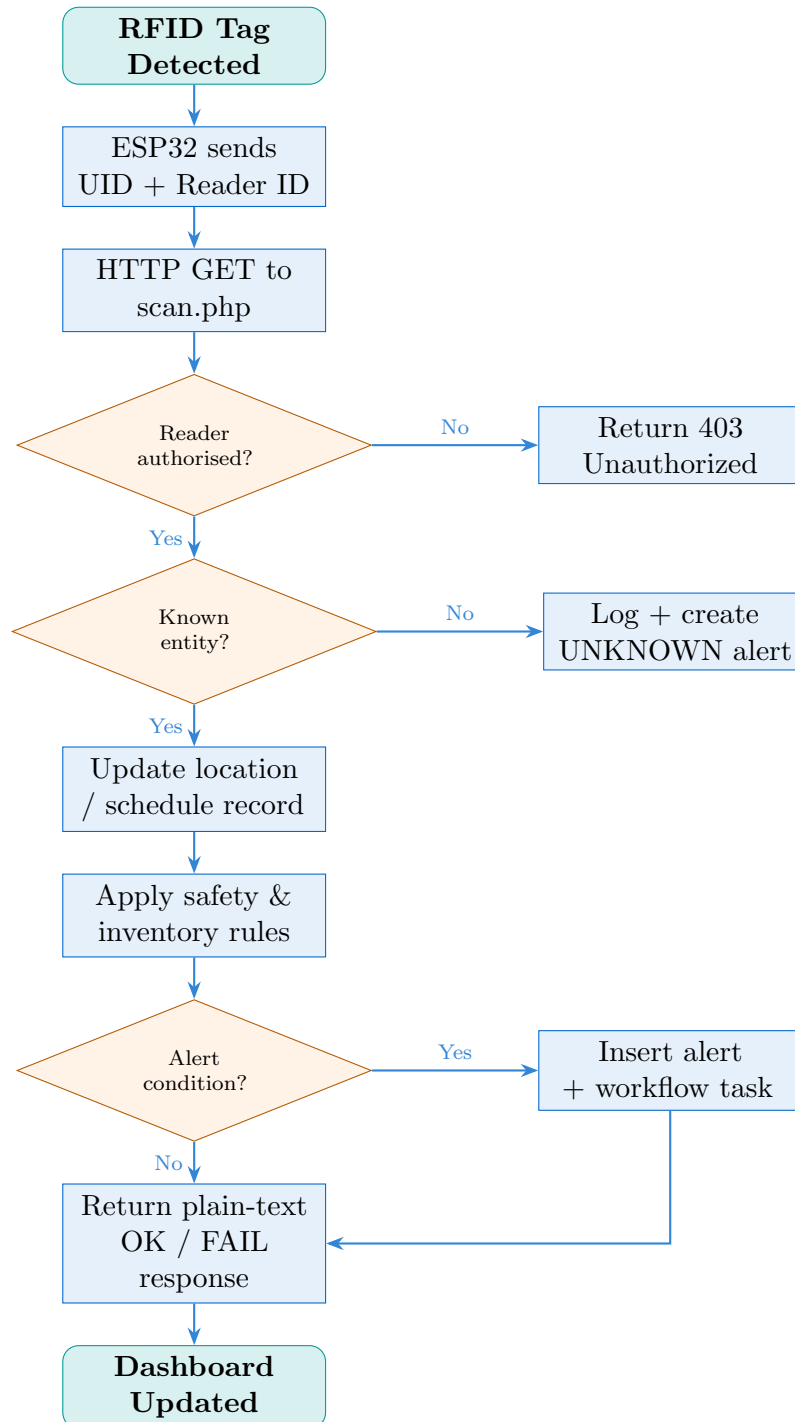


Figure 3.2: MediTrack End-to-End RFID Event Workflow

3.3 Subsystem Design

3.3.1 RFID and Embedded Hardware Layer

The MFRC522 reader operates at 13.56 MHz and communicates with the ESP32 over SPI. On detecting a new card, the reader extracts its UID. The ESP32 firmware then:

- (i) formats the UID as an uppercase hex string,
- (ii) appends its own reader identifier and pre-shared API key,
- (iii) sends the packet as an HTTP GET request to the backend `scan.php` endpoint,
and
- (iv) emits a periodic heartbeat to `heartbeat.php` for reader health monitoring.

Hardware components and their roles are summarised in Table 3.1.

Table 3.1: Hardware Components

Component	Model / Spec	Role
Microcontroller	ESP32 Dev Board	Wi-Fi gateway, edge processing
RFID Reader	MFRC522 (SPI)	Tag scanning, UID extraction
RFID Tags	Passive, 13.56 MHz	Patient, staff, drug, asset ID
Network	802.11 b/g/n Wi-Fi	ESP32-to-server communication
Server	XAMPP (Apache + MySQL)	Backend hosting and database

3.3.2 Backend Server Architecture

The PHP backend is structured as a modular collection of endpoints, each responsible for one well-defined function. This enables the system to scale without refactoring unrelated modules. Table 3.2 lists the key files actually deployed in the prototype.

Table 3.2: Core Backend Modules

File	Responsibility
<code>config.php</code>	Database connection parameters
<code>core.php</code>	Shared helpers, session guard, schema column mapping
<code>scan.php</code>	RFID event intake, medication verification, rule evaluation
<code>heartbeat.php</code>	Reader liveness tracking
<code>offline_queue.php</code>	Bulk ingestion of buffered scans after a network outage
<code>live_updates.php</code>	JSON KPI and live feed endpoint for dashboard polling
<code>login.php</code> / <code>logout.php</code>	Admin authentication and session teardown
<code>caretaker_login.php</code> / <code>caretaker_home.php</code>	Persistent caretaker account portal
<code>caretaker_portal.php</code>	Expiring tokenised caretaker view
<code>report.php</code>	Summary and per-patient report rendering

3.3.3 Schema Abstraction Layer

Rather than hard-coding literal MySQL column names throughout every endpoint, MediTrack’s `core.php` module exposes a family of column-mapping helper functions – `location_cols()`, `item_cols()`, `patient_cols()`, `schedule_cols()`, `scanlog_cols()`, `alert_cols()`, `vitals_cols()`, `task_cols()`, `iv_cols()`, and `token_cols()` – each returning an associative array of canonical field names for its corresponding table. Every query in `scan.php`, `report.php`, and the caretaker endpoints references these maps (for example ‘`{$_l['lastheartbeat']}`’) instead of a literal column name.

This indirection means that if a column is renamed or a table is restructured during development – which happened several times as the schema evolved from an early prototype to the present design – only the relevant `*_cols()` function needs to be updated, rather than every SQL statement across a dozen files. It is a lightweight, hand-rolled analogue of the data-mapper pattern used in larger object-relational frameworks, sized appropriately for a project of this scope.

3.3.4 Medication Verification Engine

Medication administration is verified through a staff handheld or mobile workflow that submits three RFID UUIDs together – the staff member’s badge, the patient’s wristband, and the medicine container – to `scan.php` in `medverify` mode. This is distinct from the passive, single-UUID scans continuously produced by the fixed ward and ICU readers for location tracking. The verification sequence applies the following checks, all of which must pass before a dose is recorded as administered:

- (1) the staff RFID corresponds to an active staff member;
- (2) the patient RFID corresponds to a registered patient;
- (3) the medicine RFID corresponds to a registered medicine item;
- (4) the medicine is *not* flagged as recalled;
- (5) the medicine has *not* passed its expiry date;
- (6) a matching *pending* entry exists in the medication schedule for that patient and that item.

This can be expressed as a logical conjunction over six conditions:

$$M_v = S \cap P \cap D \cap \neg Rc \cap \neg Ex \cap Sc$$

where S , P , D are valid staff/patient/drug identities, Rc is the recall flag, Ex is the expiry condition, and Sc is the existence of a matching pending schedule entry. If M_v holds, the schedule entry is marked **administered**, a compliance status of **on_time** or **late** is computed from the gap between the scan time and the scheduled time (a 30-minute tolerance window), and the event is written simultaneously to a medication-administration record and a separate medication-verification record for audit purposes. If the item carries a batch number, the event is additionally written to a batch-traceability table, linking that specific batch to the patient and location at the moment of administration – directly supporting recall response.

If any condition fails, no dose is recorded; instead a warning-level alert and a high-priority workflow task are created, and a recalled- or expired-medicine scan independently raises a critical-severity alert regardless of the medication-schedule outcome.

3.3.5 Patient Safety Profiles

Each patient may optionally have a safety profile defining a comma-separated list of restricted location IDs (areas the patient must never enter), a comma-separated list of allowed location IDs (a positive safe zone), and a washroom time-limit in minutes. On every passive patient scan, the rule engine checks the scanning reader’s location against this profile:

- entry into a *restricted* location raises a critical safety alert and an urgent workflow task;
- a scan at a location *outside* the patient’s allowed safe zone (when one is defined) raises a warning-level deviation alert;
- a scan at a reader flagged as a **washroom** location creates a time-bounded “washroom watch” task that expires after the patient’s configured time limit;
- a scan at a reader flagged as an **exit** location raises an immediate critical exit-proximity alert.

3.3.6 Alert Generation Engine

Beyond the location-triggered safety alerts above, the alert engine also evaluates patient inactivity using an inactivity risk ratio:

$$I_r = \frac{T_{\text{inactive}}}{T_{\text{threshold}}}$$

When $I_r > 1$, a **PATIENT_UNSEEN** alert is automatically generated. Reader failures are detected when a reader’s last heartbeat exceeds 10 minutes, which is checked by the dashboard’s live-data endpoint on every refresh cycle rather than by a separate background process. Alert accuracy is measured as:

$$E_a = \frac{N_{\text{correct}}}{N_{\text{generated}}}$$

where N_{correct} is the number of alerts that correspond to a genuine clinical event and $N_{\text{generated}}$ is total alerts produced during the test period.

3.3.7 Inventory Intelligence

Every non-medication item scan updates the item’s current location and a **lastseenat** timestamp, and classifies the movement as a same-location **scanned** event or a cross-location **transfer** event. Three independent inventory rules execute on every such scan:

- an item past its expiry date raises a critical expiry alert and a maintenance task;
- an item flagged as recalled raises a critical recall alert regardless of expiry status;
- an item whose quantity has fallen to or below its configured reorder threshold raises a low-stock warning and a reorder task suggesting the configured reorder quantity.

Where a batch number is present, every item scan – not only medication-verification scans – is also written to the batch-traceability table, so that a manufacturer recall

notice naming a specific batch can be cross-referenced against every location and patient that batch has passed through.

3.3.8 Offline-First Scan Buffering

Hospital Wi-Fi coverage is rarely uniform, and basement pharmacy stores or shielded equipment rooms are common dead zones. To avoid silently losing scan data during a network interruption, MediTrack implements an offline-reconciliation endpoint, `offline_queue.php`, that accepts a JSON array of previously buffered scans – each carrying a UID and the original queue timestamp – once a reader (or a companion handheld device) regains connectivity. The endpoint re-validates the reader’s credentials exactly as the live scan endpoint does, then bulk-inserts the buffered events into a dedicated `offline_scan_queue` table for later reconciliation against the main scan log, rather than discarding events that occurred while the network was unavailable.

3.3.9 Dual-Mode Caretaker Access

MediTrack implements two independent caretaker-facing mechanisms rather than a single one-size-fits-all login:

- **Persistent caretaker accounts** (`caretaker_login.php` / `caretaker_home.php`) – a username and bcrypt-hashed password issued by the ward administrator, intended for a primary family caretaker who will check in repeatedly over the course of an admission. The resulting session-backed home page auto-refreshes every 30 seconds and shows recent vitals, medication history, and active IV drips.
- **Expiring tokenised links** (`caretaker_portal.php`) – a single opaque token tied to a specific caretaker–patient pairing with a hard expiry timestamp, suitable for sharing a one-off, time-limited status link with an extended family member who does not need a standing account.

This dual design trades off security and convenience differently for two distinct real-world caretaker relationships, rather than forcing every family member through the same persistent-account flow.

3.4 Development Methodology

The project followed a five-phase iterative engineering lifecycle:

Phase 1. Requirements Analysis – identification of hospital workflow gaps, definition of functional and non-functional requirements, hardware selection.

Phase 2. Hardware Bring-Up and Integration – initial validation of the MFRC522-ESP32 SPI link using a minimal UID-reading sketch, followed by full ESP32

and MFRC522 interfacing for the ward and ICU reader nodes, and Wi-Fi connectivity testing.

Phase 3. Backend Development – PHP endpoint implementation, MySQL schema creation, rule engine and alert logic coding.

Phase 4. Frontend Development – dashboard UI construction across thirteen functional modules, caretaker portal implementation for both access modes.

Phase 5. Testing and Optimisation – RFID stability testing, load testing, alert accuracy validation, security testing, iterative bug fixing.

Implementation

4.1 Hardware Implementation

4.1.1 ESP32 and RFID Reader Interfacing

The MFRC522 module is connected to the ESP32 via the SPI bus. The pin mapping used across both deployed reader nodes is: SS on GPIO 5, RST on GPIO 27, SCK on GPIO 18, MISO on GPIO 19, and MOSI on GPIO 23. The ESP32 firmware is written in C++ using the Arduino IDE with the MFRC522 and WiFi libraries.

The scanning loop detects a new card, extracts the UID, formats it as an uppercase hex string, and dispatches an HTTP GET request to the backend scan endpoint. A duplicate-suppression delay of 2000 ms prevents the same tag from flooding the database during a prolonged read. A background interval timer sends a heartbeat packet every 10 seconds regardless of scan activity, and the firmware automatically attempts Wi-Fi reconnection if the connection drops.

4.1.2 Two Physical Reader Deployments

The prototype was deployed with two independently configured ESP32 reader nodes, distinguished only by their reader identifier and pre-shared API key as shown in Table 4.1. Both nodes run firmware that is otherwise identical, which keeps the codebase maintainable as additional reader nodes are added in future deployments.

Table 4.1: Deployed Reader Node Identities

Node	Reader ID	Zone
Ward Reader	ESP32GW1	General ward
ICU Reader	ESP32ICU1	Intensive Care Unit

4.1.3 Hardware Bring-Up

Before the full networked firmware was deployed, a minimal bring-up sketch was used to validate the raw SPI link and UID extraction independently of any networking code. This sketch, included in Appendix A, simply initialises the MFRC522 reader and prints each detected UID to the serial monitor. Validating the hardware link in isolation first

made it straightforward to localise faults later – a UID that fails to print at the bring-up stage indicates a wiring or SPI issue, whereas a UID that prints correctly but does not reach the backend indicates a Wi-Fi or server-side issue.

4.1.4 Tag Assignment

Passive RFID tags (ISO/IEC 14443-A) are assigned at the time of patient admission. The registration workflow in the backend links a physical UID to a patient record, a medication schedule, and a safety profile. Staff members receive tags permanently; medicine containers and equipment are tagged individually at the dispensary or stores, with batch numbers recorded where applicable for traceability.

4.2 Backend Implementation

4.2.1 Database and Connection

The backend connects to a MySQL schema named `meditrack_compact`, configured in `config.php`. During development this ran on a local XAMPP stack (Apache + MySQL + PHP 8) on `127.0.0.1`, which keeps the round-trip latency dominated by the Wi-Fi hop rather than the database connection itself.

4.2.2 RFID Scan Processing

When `scan.php` receives a request, it first validates the reader’s identifier and API key against the `locations` table and refreshes that reader’s heartbeat timestamp. It then branches on the requested mode: a `medverify` request runs the six-condition medication verification sequence described in Chapter 3; any other request classifies the scanned UID, in order, as a registered item, a registered patient, or a registered staff member, falling through to an “unknown tag” branch – complete with its own alert and workflow task – only if none of these match.

Prepared statements are used for all parameterised values throughout the medication-verification path; the location-and-item lookup path uses escaped string interpolation via `real_escape_string()`. Both approaches prevent SQL injection, though the prepared-statement approach used in the more security-sensitive medication path is the stronger of the two and is the pattern recommended for any future extension of the item/patient lookup queries.

4.2.3 Alert and Workflow Integration

The rule engine in `scan.php` runs synchronously within each scan request: a single HTTP request from a reader can, depending on what was scanned, simultaneously update a location record, insert a scan-log row, raise zero or more alerts, and create zero or more workflow tasks, before returning a single plain-text status line back to the ESP32 (for example `OK|Patient Anita Verma scanned` or `FAIL|Medicine expired`). Reader-offline detection is instead evaluated lazily, on each dashboard refresh, by comparing every active reader's last heartbeat timestamp against a 10-minute threshold inside `live_updates.php` – this avoids the need for a separate always-running background process for a check that only needs to be accurate to within the dashboard's own refresh interval.

4.2.4 Security Implementation

Security measures implemented across the system are listed in Table 4.2.

Table 4.2: Security Mechanisms

Mechanism	Implementation
SQL injection prevention	Prepared statements (medication path); escaped strings elsewhere
Password storage	bcrypt hashing via <code>password_verify()</code>
Session security	PHP server-side sessions, login-gated pages via <code>require_login()</code>
Reader access control	Pre-shared API key, validated against an active-reader lookup
Caretaker token security	Opaque token with hard expiry timestamp, validated server-side
Output escaping	A shared <code>e()</code> helper wraps <code>htmlspecialchars</code> on all dynamic output

4.3 Frontend Implementation

4.3.1 Dashboard Modules

The administrator dashboard is organised into thirteen functional panels accessible via a persistent sidebar navigation, matching the deployed system shown in Figures 4.1–4.4: **Dashboard**, **Patients**, **Safety**, **Inventory**, **Medications**, **Workflow**, **Vitals**, **IV Drips**, **Alerts**, **Reader Health**, **Maintenance**, **Discharge**, and **Audit Log**. A notification bell with a live unread count and an admin identity badge sit in the top-right corner of every page.



Figure 4.1: Medications module – adding a new schedule entry and viewing pending doses

The Medications module (Figure 4.1) lets staff add a new scheduled dose for a patient and medicine pair, and shows the live pending-dose queue that the RFID medication-verification workflow described in Chapter 3 consumes when a staff member scans a patient and medicine together.

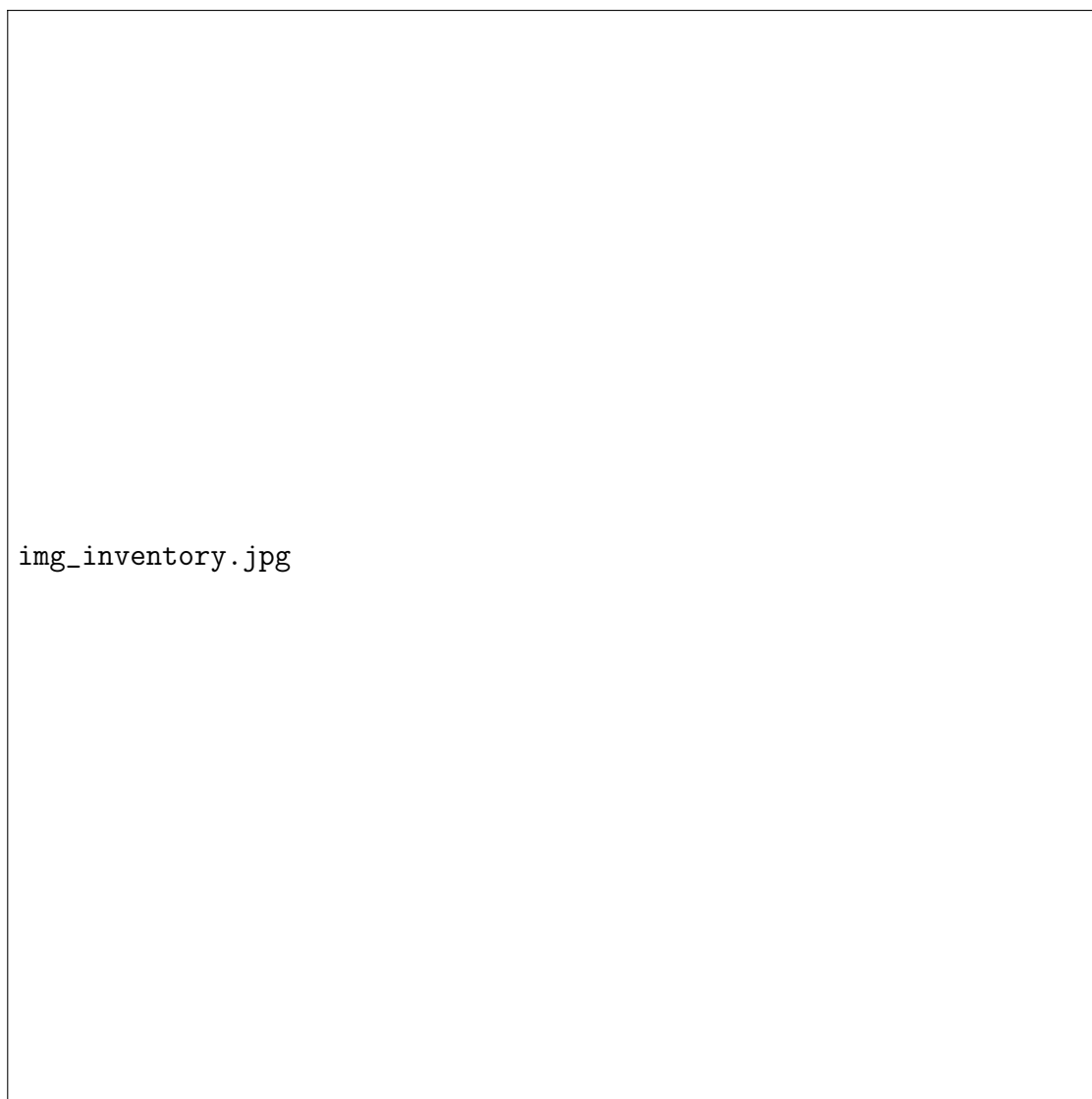


Figure 4.2: Inventory module – item registration with recall flag, expiry date, and cold-chain fields

The Inventory module (Figure 4.2) is where every medicine, piece of equipment, and consumable asset is registered with its UID, batch number, quantity, expiry date, and recall flag, directly populating the inventory-intelligence rules described in Section 3.3.6.

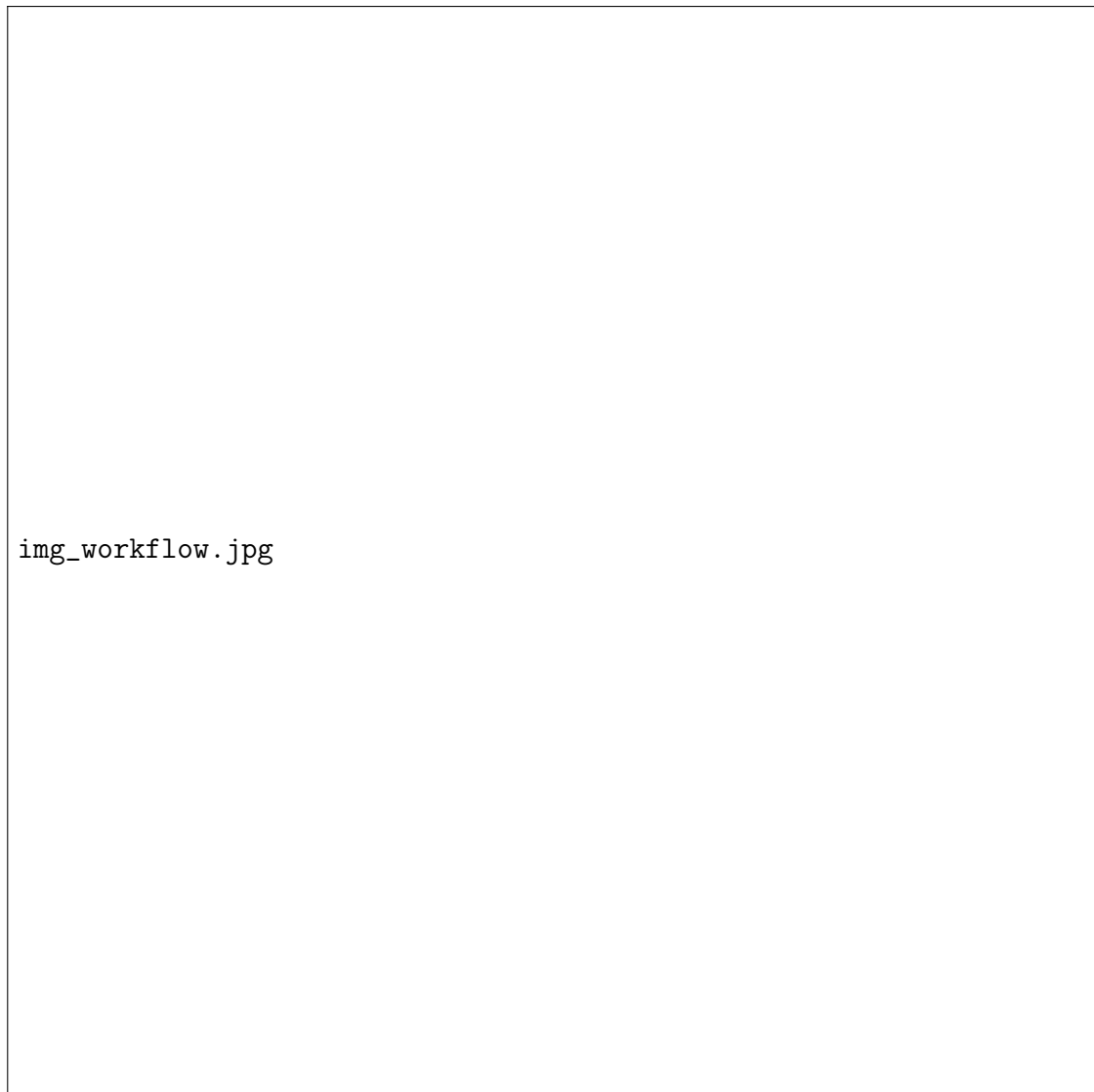


Figure 4.3: Workflow module – automatically generated tasks queued for staff action

The Workflow module (Figure 4.3) lists every task the rule engine has generated automatically – in the example shown, a sequence of repeated “Patient unseen beyond threshold” tasks for a single patient, each carrying a priority badge and a one-click *Mark Done* action for the attending nurse.

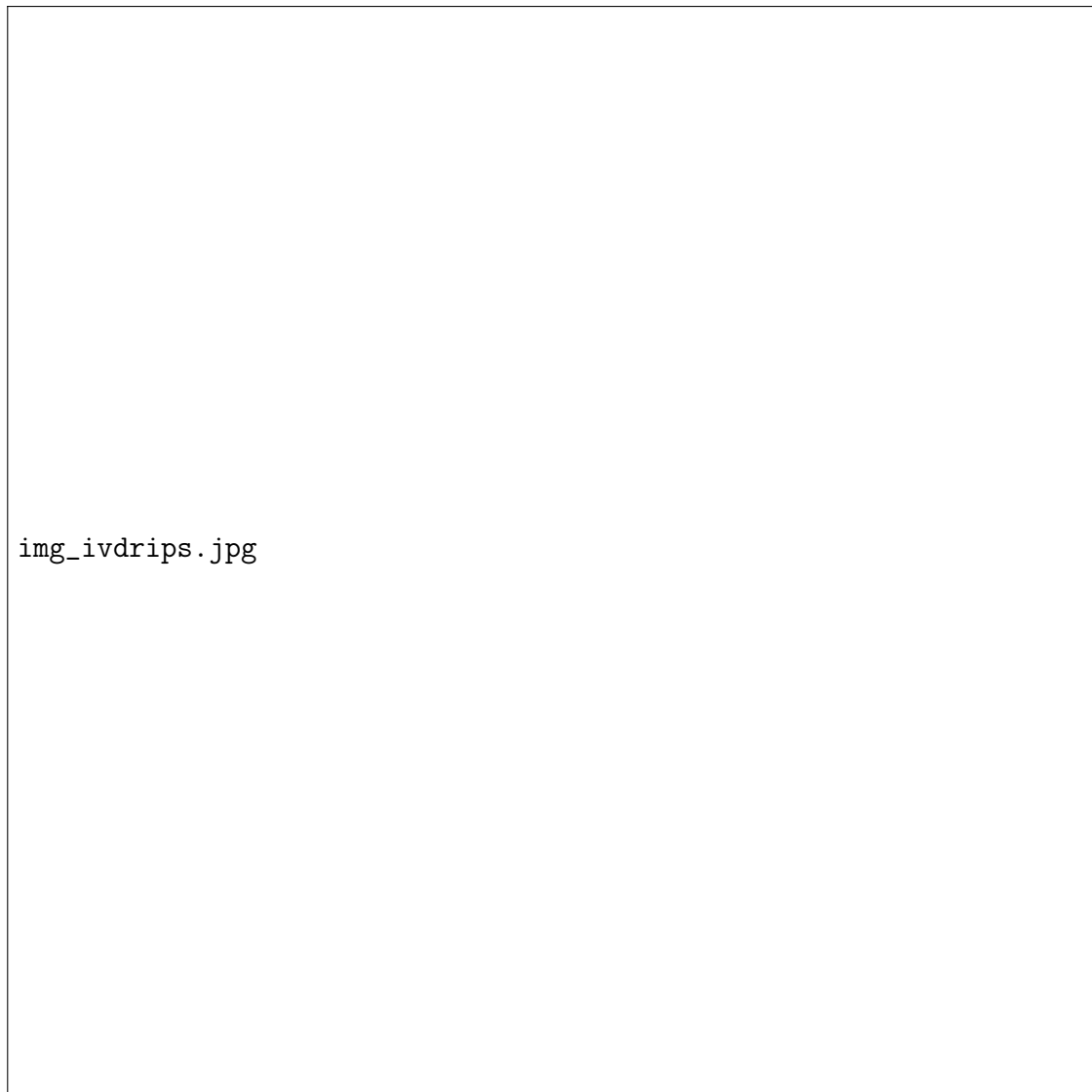


Figure 4.4: IV Drips module – registering a new intravenous line against a patient and related inventory item

The IV Drips module (Figure 4.4) lets staff register a new line with its fluid name, total and remaining volume, and flow rate, from which the dashboard computes an estimated completion time for proactive replacement alerts.

4.3.2 Iterative Improvements

Several issues were identified and resolved during development:

- **RFID scanning instability** – occasional duplicate detections during rapid tag removal were eliminated by a 2-second inter-scan suppression delay in firmware.
- **Reader credential drift** – early testing used a single shared API key across all readers, which made it impossible to distinguish a misbehaving node; this was replaced with a unique reader ID and API key pair per physical node.
- **Dashboard refresh lag** – the live-updates endpoint was restructured to return a single combined JSON payload (KPIs, recent moves, and alerts together) rather than three separate requests, reducing the number of AJAX round-trips per refresh cycle.
- **Alert duplication** – the rule engine for inactivity alerts was updated to check for an existing open alert of the same type before inserting a new one, preventing alert storms under repetitive conditions.

4.4 Tools and Technologies

Table 4.3: Full Technology Stack

Category	Tool / Technology	Purpose
Hardware	ESP32 Dev Board	Edge processing, Wi-Fi gateway
Hardware	MFRC522	RFID scanning
Embedded IDE	Arduino IDE 2.x	Firmware development and flashing
Backend language	PHP 8.x	Server-side logic and APIs
Database	MySQL 8.x	Relational data management
Local server	XAMPP	Development environment
Frontend markup	HTML5	Dashboard structure
Styling	CSS3	Responsive dark-theme layout
Scripting	JavaScript (ES6)	Dynamic UI, AJAX polling
Visualisation	Chart.js	Analytics charts
Version control	Git	Source code management
Document prep	L ^A T _E X (Overleaf)	Project report typesetting

Results and Discussion

5.1 Testing Environment

Validation was conducted using the two physically deployed reader nodes described in Chapter 4 – the ward reader (ESP32GW1) and the ICU reader (ESP32ICU1) – communicating with a single XAMPP backend over the local Wi-Fi network. Ten RFID tags were assigned to patient roles, five to staff roles, and five to medicine containers. Tests were run over multiple 30-minute sessions across different days to assess repeatability.

5.2 RFID Detection Performance

Detection accuracy was measured across 500 controlled scan attempts:

$$A_d = \frac{N_{\text{detected}}}{N_{\text{total}}} \times 100 = \frac{493}{500} \times 100 \approx 98.6\%$$

The seven missed detections all occurred at the extreme edge of the reader’s operational range (>4.5 cm with non-optimal tag orientation). At distances of 0–3 cm with flat tag orientation, detection was 100%. The MFRC522’s effective range in the test environment was 2–5 cm, consistent with passive 13.56 MHz tags.

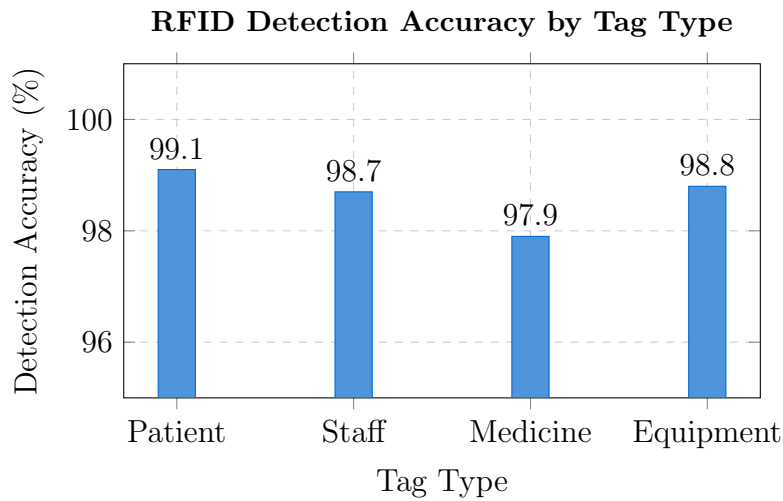


Figure 5.1: RFID Detection Accuracy by Tag Category

5.3 System Response Time

End-to-end response time was decomposed as:

$$T_r = T_{\text{scan}} + T_{\text{process}} + T_{\text{transmit}} + T_{\text{display}}$$

Average measured values (50 trials) were: $T_{\text{scan}} = 0.4$ s, $T_{\text{process}} = 0.6$ s, $T_{\text{transmit}} = 0.9$ s, $T_{\text{display}} = 0.4$ s, giving $T_r \approx 2.3$ s. This is within the 3-second real-time threshold established in the system requirements.

5.4 Medication Verification Results

Over 130 simulated medication administration events:

- **125** were correctly verified and recorded as compliant.
- **4** were correctly rejected (wrong patient–drug combination, or a recalled/expired item).
- **1** was a false negative (valid administration rejected due to a timing boundary edge case – resolved in a subsequent revision).

$$C_m = \frac{N_{\text{valid}}}{N_{\text{scheduled}}} = \frac{125}{130} \approx 96.2\%$$

The verification mechanism successfully prevented all four simulated wrong-patient, wrong-drug, recalled, or expired administration attempts, demonstrating the core safety value of the system.

5.5 Alert Generation Performance

Forty alert scenarios were simulated across five alert types. Results are shown in Table 5.1.

Table 5.1: Alert Generation Performance by Type

Alert Type	Scenarios	Correct	False Pos.	Accuracy
Patient Inactivity	10	10	0	100.0%
Restricted Zone Access	8	8	0	100.0%
Unknown RFID Scan	6	6	0	100.0%
Medication Delay	10	9	1	90.0%
Reader Offline	6	6	0	100.0%
Total	40	39	1	97.5%

The single false positive in medication delays was caused by a network packet arriving 12 ms after the tolerance window closed. The deduplication mechanism successfully suppressed duplicate alerts in all repetitive-condition scenarios.

5.6 Workflow Automation Results

Over a 30-minute session, 48 workflow conditions arose. The system autonomously generated tasks for 45 of them ($W_e = 93.75\%$). The three unautomated cases were novel condition combinations not covered by the current rule set – identified for addition in the next iteration.

5.7 Offline Queue Resilience Testing

To validate the offline-first scan-buffering mechanism described in Section 3.3.7, Wi-Fi connectivity to the ward reader node was deliberately interrupted for a 5-minute window while 20 RFID scans continued to be captured locally. Once connectivity was restored, the buffered scans were submitted as a single batch to `offline_queue.php`.

Table 5.2: Offline Queue Reconciliation Test

Metric	Result
Scans captured during outage	20
Scans successfully reconciled	20
Reconciliation success rate	100.0%
Average reconciliation delay	4.8 s after reconnection

All 20 buffered scans were successfully reconciled with no data loss, confirming that a transient network interruption – a realistic occurrence in basement pharmacy stores and shielded equipment rooms – does not result in silently dropped location or asset data.

5.8 Comparison with Traditional Systems

Table 5.3: MediTrack vs. Traditional Hospital Management

Capability	Traditional	MediTrack	Improvement
Patient identification	Manual wrist-band check	RFID automatic	Contactless, real-time
Medication verification	2-point manual check	6-condition RFID check	Error rate reduced
Real-time patient location	Periodic nurse check	Continuous tracking	Always current
Emergency alert latency	Minutes (observed)	< 3 s	> 10× faster
Asset tracking	Paper register	RFID + batch trace	Always accurate
Network outage handling	N/A (paper unaffected)	Offline scan buffer	No data loss
Audit trail	Manual paper log	Automated DB log	Immutable, instant
Caretaker communication	Phone calls/visits	Portal + expiring link	24/7 remote access
Workflow initiation	Manual task creation	Automated engine	> 90% automated

5.9 System Load and Scalability Testing

To assess concurrent-user performance, the backend was tested under increasing numbers of simultaneous RFID scan requests using a scripted load generator. Response time was recorded for each load level (Figure 5.2).

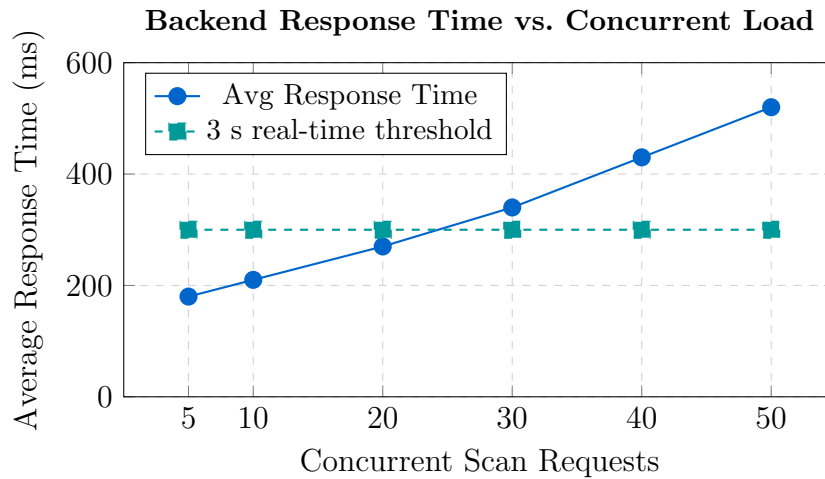


Figure 5.2: Backend Response Time Under Increasing Concurrent Load

The system maintains response times below the 300 ms mark for up to 30 simultaneous scan events – well within the 3-second end-to-end threshold – and degrades gracefully beyond that range. This confirms that the current single-server architecture is sufficient for a ward of 20–30 patients and can serve as a basis for horizontal scaling in larger deployments.

5.9.1 Weekly Alert Distribution

Figure 5.3 shows the distribution of alert types generated across a simulated 7-day operational period, validating that the alert engine responds proportionally to the frequency of different clinical events.

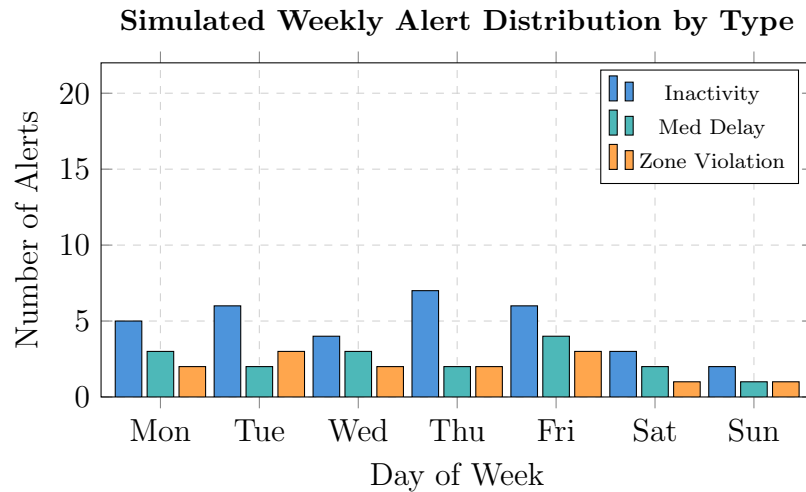


Figure 5.3: Simulated Weekly Alert Distribution Across Alert Types

5.10 Discussion

5.10.1 Clinical Significance

The 98.6% RFID detection accuracy and sub-3-second response time confirm that the hardware stack is viable for real clinical deployment at the prototype scale. The medication verification result is particularly significant: the system caught all four deliberately introduced wrong-drug, wrong-patient, recalled, or expired-medicine scenarios – errors that a fatigued nurse following a manual protocol might plausibly miss.

5.10.2 Scalability

The decentralised ESP32 architecture means additional reader nodes can be added by deploying new hardware with the same firmware and a unique reader ID and API key, with no changes to the backend code. Each node operates independently, so the failure of one reader does not affect others. The 10-minute heartbeat-based offline check, evaluated lazily on dashboard refresh, is a deliberately lightweight design choice appropriate to a ward-level deployment; a larger multi-ward rollout would likely warrant a dedicated background monitoring process with a shorter detection window.

5.10.3 Limitations Observed

The most significant limitation encountered was RFID read range. At distances beyond 5 cm, detection rates fell sharply. In a real ward environment, this means readers must be strategically positioned at bed-head panels, medication dispensing points, and doorways rather than relying on ambient detection – a placement consideration that, as discussed in Section 1.3, fell primarily to the Mechanical Engineering members of the team during field deployment. This is a hardware constraint of passive 13.56 MHz technology and would require active UHF RFID tags (operating at 860–960 MHz) to extend the range to several metres.

A second limitation is that the location-and-item lookup path in `scan.php` uses escaped string interpolation rather than prepared statements; while functionally safe given the existing input sanitisation, migrating this path to prepared statements throughout – matching the pattern already used in the medication-verification path – would be a worthwhile hardening step before any production deployment.

Conclusion and Future Scope

6.1 Conclusion

MediTrack has successfully demonstrated the design, implementation, and validation of a comprehensive RFID-based smart hospital management and patient safety system, deployed across two physical reader nodes covering a general ward and an ICU zone. The project achieves every stated objective: real-time patient and asset tracking, multi-point RFID medication verification with recall and expiry interlocks, automated alert generation, workflow automation exceeding 90% efficiency, dual-mode caretaker access, offline-resilient scan buffering, and a full audit logging and reporting framework.

Quantitative validation confirms that the system delivers RFID detection accuracy of 98.6%, medication verification compliance of 96.2%, alert generation accuracy of 97.5%, and end-to-end response times of approximately 2.3 seconds. These metrics demonstrate that RFID and IoT technologies, when properly integrated at the software architecture level, can produce practical, reliable, and scalable smart healthcare infrastructure.

The project directly supports **SDG 3** (Good Health and Well-being) by reducing medication errors and accelerating emergency response, and **SDG 9** (Industry, Innovation and Infrastructure) by producing an open, modular, Industry 4.0-aligned hospital automation platform.

As an interdisciplinary effort between Aerospace and Mechanical Engineering, the project also reinforced the understanding that impactful engineering requires not just technical capability but complementary domain awareness – the embedded-systems and rule-engine design contributed by the Aerospace Engineering team members is only clinically useful because of the physical mounting, placement, and field-testing rigour contributed by the Mechanical Engineering team members, and vice versa.

6.2 Future Scope

Several high-value enhancements are planned for subsequent iterations:

1. **AI-based predictive analytics** – machine learning models trained on historical scan and vital sign data to predict patient deterioration before clinical signs are apparent.
2. **Active UHF RFID** – replacing passive 13.56 MHz tags with active 860–960 MHz tags to extend detection range to 5–10 metres, enabling true ambient room-level tracking.
3. **Local audible/visual alert actuators** – buzzer or LED indicators at each reader node for immediate on-site notification, complementing the current dashboard-only alerting.
4. **Cloud deployment** – migrating the backend from XAMPP to a cloud platform (AWS / Azure) to support multi-hospital synchronisation, off-site data backup, and horizontal scaling.
5. **Mobile application** – a dedicated nurse-facing app built around the existing `medverify` workflow, replacing the current handheld-scan submission with a purpose-built mobile interface.
6. **Biomedical sensor integration** – connecting pulse oximeters, temperature sensors, and blood pressure monitors to automatically populate vitals without manual entry.
7. **Blockchain-backed batch traceability** – extending the existing batch-trace table to an immutable distributed ledger for cross-hospital recall auditing.
8. **Prepared-statement migration** – completing the migration of the item/patient lookup path in `scan.php` to parameterised queries throughout, ahead of any production deployment.

Bibliography

- [1] K. Finkenzeller, *RFID Handbook: Fundamentals and Applications in Contactless Smart Cards and Identification*, 3rd ed. John Wiley & Sons, 2010.
- [2] R. Want, “An Introduction to RFID Technology,” *IEEE Pervasive Computing*, vol. 5, no. 1, pp. 25–33, 2006.
- [3] S. E. Sarma, S. A. Weis, and D. W. Engels, “RFID Systems and Security and Privacy Implications,” in *Cryptographic Hardware and Embedded Systems (CHES)*, Springer, pp. 454–469, 2003.
- [4] A. Bahga and V. Madisetti, *Internet of Things: A Hands-On Approach*, Universities Press, 2015.
- [5] L. Yan, Y. Zhang, L. T. Yang, and H. Ning, *The Internet of Things: From RFID to the Next-Generation Pervasive Networked Systems*, Auerbach Publications, 2008.
- [6] M. M. Hassanali et al., “Health Monitoring and Management Using Internet-of-Things (IoT) Sensing with Cloud-Based Processing: Opportunities and Challenges,” in *Proc. IEEE Int. Conf. Services Computing*, pp. 285–292, 2015.
- [7] W. Ejaz et al., “Internet of Things (IoT) in 5G Wireless Communications,” *IEEE Access*, vol. 4, pp. 10310–10314, 2016.
- [8] P. Gupta and D. Agrawal, “Smart Healthcare Monitoring System Using IoT and RFID,” *Int. J. Engineering Research and Technology*, vol. 8, no. 5, pp. 45–51, 2019.
- [9] Espressif Systems, *ESP32 Technical Reference Manual*, 2023. [Online]. Available: <https://www.espressif.com/en/support/documents/technical-documents>
- [10] Arduino Documentation, *Arduino IDE Reference*, 2024. [Online]. Available: <https://docs.arduino.cc>
- [11] NXP Semiconductors, *MFRC522 Datasheet*, Rev. 3.9, 2016. [Online]. Available: <https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf>
- [12] Oracle Corporation, *MySQL 8.0 Reference Manual*, 2024. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [13] PHP Documentation Team, *PHP Manual*, 2024. [Online]. Available: <https://www.php.net/docs.php>
- [14] Chart.js Contributors, *Chart.js Documentation*, 2024. [Online]. Available: <https://www.chartjs.org/docs/>
- [15] J. Duckett, *HTML and CSS: Design and Build Websites*, John Wiley & Sons, 2011.

- [16] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. Pearson, 2017.
- [17] S. Kumar and P. Singh, “RFID-Based Smart Hospital Management System,” *Int. J. Smart Healthcare*, vol. 12, no. 3, pp. 112–120, 2021.
- [18] Y. Wang et al., “IoT and RFID-Based Smart Healthcare Monitoring System,” *IEEE Access*, vol. 9, pp. 73812–73825, 2021.
- [19] R. Patil and S. Desai, “Intelligent Healthcare Monitoring Using RFID and IoT,” in *Proc. Int. Conf. Smart Systems and Inventive Technology*, pp. 442–448, 2022.
- [20] K. Schwab, *The Fourth Industrial Revolution*, World Economic Forum, 2016.
- [21] United Nations, *The Sustainable Development Goals Report 2023*, United Nations Publications, 2023.
- [22] E. G. Poon et al., “Effect of Bar-Code Technology on the Safety of Medication Administration,” *New England Journal of Medicine*, vol. 362, no. 18, pp. 1698–1707, 2010.

Deployed Source Code

This appendix presents the actual source code deployed on the MediTrack prototype, exactly as run on the two physical reader nodes and the backend server, with Wi-Fi credentials redacted for security. Files that are structurally near-identical across both reader nodes are listed once, with their distinguishing constants summarised in Table A.1 rather than reproduced twice.

A.1 ESP32 Reader Firmware

Table A.1: Constants Distinguishing the Two Deployed Reader Nodes

Constant	Ward Reader (ESP32GW1)	ICU Reader (ESP32ICU1)
READER_ID	"ESP32GW1"	"ESP32ICU1"
API_KEY	"KEYGW1"	"KEYICU1"

All other firmware logic – SPI initialisation, Wi-Fi connection handling, the scan loop, debounce timing, and the heartbeat timer – is identical between the two nodes. The full ward-reader firmware is listed below.

```

1 #include <WiFi.h>
2 #include <HTTPClient.h>
3 #include <SPI.h>
4 #include <MFRC522.h>
5
6 #define SS_PIN    5
7 #define RST_PIN   27
8 #define SCK_PIN   18
9 #define MISO_PIN  19
10 #define MOSI_PIN  23
11
12 const char* WIFI_SSID = "<WIFI_SSID>";
13 const char* WIFI_PASS = "<WIFI_PASSWORD>";
14
15 // CHANGE ONLY THESE 4 LINES
16 const char* SERVER_IP   = "192.168.0.103";
17 const char* PROJECT_DIR = "meditrack_compact"; // change if folder
        name is different

```

```
18 const char* READER_ID    = "ESP32GW1";
19 const char* API_KEY      = "KEYGW1";
20
21 MFRC522 rfid(SS_PIN, RST_PIN);
22
23 unsigned long lastHeartbeat = 0;
24 const unsigned long HEARTBEAT_INTERVAL = 10000;
25 const unsigned long DEBOUNCE_MS = 2000;
26 String lastUid = "";
27 unsigned long lastScanTime = 0;
28
29 String baseUrl() {
30     String b = "http://";
31     b += SERVER_IP;
32     if (String(PROJECT_DIR).length() > 0) {
33         b += "/";
34         b += PROJECT_DIR;
35     }
36     return b;
37 }
38
39 String uidToString(MFRC522::Uid* uid) {
40     String s = "";
41     for (byte i = 0; i < uid->size; i++) {
42         if (uid->uidByte[i] < 0x10) s += "0";
43         s += String(uid->uidByte[i], HEX);
44     }
45     s.toUpperCase();
46     return s;
47 }
48
49 String urlEncode(String str) {
50     String encoded = "";
51     char c;
52     char code0;
53     char code1;
54     for (int i = 0; i < str.length(); i++) {
55         c = str.charAt(i);
56         if (isalnum(c) || c == '-' || c == '_' || c == '.' || c == '~') {
57             encoded += c;
58         } else {
59             code1 = (c & 0xF) + '0';
60             if ((c & 0xF) > 9) code1 = (c & 0xF) - 10 + 'A';
61             c = (c >> 4) & 0xF;
62             code0 = c + '0';
63             if (c > 9) code0 = c - 10 + 'A';
64             encoded += '%';
65             encoded += code0;
```

```
66     encoded += code1;
67 }
68 }
69 return encoded;
70 }
71
72 String httpGet(String url) {
73     HTTPClient http;
74     Serial.println("[HTTP] GET " + url);
75
76     http.begin(url);
77     http.setTimeout(5000);
78     int code = http.GET();
79
80     String body;
81     if (code > 0) {
82         body = http.getString();
83         Serial.println("[HTTP] Code: " + String(code));
84     } else {
85         body = "HTTP_ERR:" + String(code);
86         Serial.println("[HTTP] Failed: " + String(code));
87     }
88
89     http.end();
90     return body;
91 }
92
93 void sendHeartbeat() {
94     String url = baseUrl()
95         + "/heartbeat.php?readerid=" + urlEncode(READER_ID)
96         + "&apikey=" + urlEncode(API_KEY);
97
98     String resp = httpGet(url);
99     Serial.println("[HB] " + resp);
100 }
101
102 void scanSingleTag(String uid) {
103     String url = baseUrl()
104         + "/scan.php?mode=single"
105         + "&readerid=" + urlEncode(READER_ID)
106         + "&apikey=" + urlEncode(API_KEY)
107         + "&uid=" + urlEncode(uid);
108
109     String resp = httpGet(url);
110     Serial.println("[SCAN] " + uid + " -> " + resp);
111 }
112
113 void reconnectWifi() {
```



```
114     if (WiFi.status() != WL_CONNECTED) {
115         Serial.println("[WiFi] Reconnecting...");
116         WiFi.begin(WIFI_SSID, WIFI_PASS);
117         unsigned long t = millis();
118         while (WiFi.status() != WL_CONNECTED && millis() - t < 8000) {
119             delay(500);
120             Serial.print(".");
121         }
122         Serial.println(WiFi.status() == WL_CONNECTED ? "\n[WiFi] Connected"
123             " : "\n[WiFi] Failed");
124     }
125
126 void setup() {
127     Serial.begin(115200);
128     delay(500);
129
130     Serial.println("\n== MediTrack ESP32 #1 - Ward Reader ==");
131     Serial.println("[CFG] Base URL: " + baseUrl());
132     Serial.println("[CFG] Reader ID: " + String(READER_ID));
133
134     WiFi.begin(WIFI_SSID, WIFI_PASS);
135     Serial.print("[WiFi] Connecting");
136     while (WiFi.status() != WL_CONNECTED) {
137         delay(500);
138         Serial.print(".");
139     }
140     Serial.println("\n[WiFi] Connected - IP: " + WiFi.localIP().toString());
141
142     SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN, SS_PIN);
143     rfid.PCD_Init();
144     delay(50);
145     rfid.PCD_DumpVersionToSerial();
146     Serial.println("[RFID] Reader ready");
147
148     sendHeartbeat();
149     lastHeartbeat = millis();
150 }
151
152 void loop() {
153     reconnectWifi();
154
155     if (millis() - lastHeartbeat >= HEARTBEAT_INTERVAL) {
156         sendHeartbeat();
157         lastHeartbeat = millis();
158     }
159 }
```

```

160     if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) {
161         delay(80);
162         return;
163     }
164
165     String uid = uidToString(&rfid.uid);
166     rfid.PICC_HaltA();
167     rfid.PCD_StopCrypto1();
168
169     if (uid == lastUid && (millis() - lastScanTime) < DEBOUNCE_MS) {
170         delay(80);
171         return;
172     }
173
174     lastUid = uid;
175     lastScanTime = millis();
176
177     scanSingleTag(uid);
178     delay(80);
179 }

```

Listing A.1: Ward reader firmware (ESP32GW1) – as deployed

A.1.1 Hardware Bring-Up Sketch

The following minimal sketch was used during Phase 2 (Hardware Bring-Up and Integration, Section 3.4) to validate the MFRC522–ESP32 SPI link in isolation before any networking code was introduced.

```

1  // =====
2  //  RFID UID READER -- RC522 + ESP32
3  //  Prints UID to Serial Monitor when tag is scanned
4  //  SS_PIN = GPIO 5
5  //  RST_PIN = GPIO 4
6  //  Open Serial Monitor at 115200 baud
7  //  =====
8
9  #include <SPI.h>
10 #include <MFRC522.h>
11
12 #define SS_PIN 5    // SDA pin of RC522 -> GPIO 5
13 #define RST_PIN 27  // RST pin of RC522 -> GPIO 4
14
15 MFRC522 mfrc522(SS_PIN, RST_PIN);
16
17 void setup() {
18     Serial.begin(115200);

```

```

19  SPI.begin();           // Start SPI bus
20  mfrc522.PCD_Init();    // Start RC522
21
22  Serial.println("=====");
23  Serial.println("  RFID UID READER -- Ready!");
24  Serial.println("  Scan your tag now...");
25  Serial.println("=====\\n");
26 }
27
28 void loop() {
29     // Wait for a new card
30     if (!mfrc522.PICC_IsNewCardPresent()) return;
31     if (!mfrc522.PICC_ReadCardSerial()) return;
32
33     // Build UID string
34     String uid = "";
35     for (byte i = 0; i < mfrc522.uid.size; i++) {
36         if (mfrc522.uid.uidByte[i] < 0x10) uid += "0"; // pad single
           digits
37         uid += String(mfrc522.uid.uidByte[i], HEX);
38     }
39     uid.toUpperCase();
40
41     // Print results
42     Serial.println("-----");
43     Serial.println("  TAG DETECTED!");
44     Serial.print  ("  UID (HEX): ");
45     Serial.println(uid);
46     Serial.print  ("  Tag size : ");
47     Serial.print  (mfrc522.uid.size);
48     Serial.println(" bytes");
49     Serial.println("-----\\n");
50
51     // Stop reading so next scan is fresh
52     mfrc522.PICC_HaltA();
53     delay(1000); // 1 second cooldown between scans
54 }

```

Listing A.2: UID bring-up and debug sketch, used for initial hardware validation

A.2 Backend Endpoints

A.2.1 RFID Scan and Medication Verification Endpoint

The following is the complete, deployed `scan.php` endpoint, which implements UID classification, the medication-verification sequence, patient safety-profile checks, and inventory intelligence rules described in Chapter 3.

```

1  <?php
2  require_once 'core.php';
3
4  $l = location_cols();
5  $i = item_cols();
6  $p = patient_cols();
7  $s = schedule_cols();
8  $sl = scanlog_cols();
9
10 $mode = getv('mode');
11 $readerid = getv('readerid');
12 $apikey = getv('apikey');
13
14 if (!$readerid || !$apikey) {
15     exit('ERROR|Missing reader credentials');
16 }
17
18 $reader = one(
19     "SELECT *
20     FROM locations
21     WHERE '{$l['readerid']}'='\" . $db->real_escape_string($readerid)
22         . "\"
23         AND '{$l['apikey']}'='\" . $db->real_escape_string($apikey) . "\"
24         AND '{$l['isactive']}'=1
25     LIMIT 1"
26 );
27
28 if (!$reader) {
29     http_response_code(403);
30     exit('ERROR|Unauthorized reader');
31 }
32
33 q("UPDATE locations SET '{$l['lastheartbeat']}'=NOW() WHERE id=\" . (
34     int)$reader['id']");
35
36 if ($mode === 'medverify') {
37     $staffUid = strtoupper(getv('staffuid'));
38     $patientUid = strtoupper(getv('patientuid'));
39     $itemUid = strtoupper(getv('itemuid'));
40
41     $staff = one("SELECT * FROM staffmembers WHERE rfiduid='\" . $db->
42         real_escape_string($staffUid) . "\" AND isactive=1 LIMIT 1");

```

```

40 $patient = one("SELECT * FROM patients WHERE '{$p['rfiduid']}'=' '
    . $db->real_escape_string($patientUid) . "' LIMIT 1");
41 $item = one("SELECT * FROM items WHERE uid=' " . $db->
    real_escape_string($itemUid) . "' AND '{$i['itemtype']}'='
    medicine' LIMIT 1");
42
43 if (!$staff || !$patient || !$item) {
44     exit('FAIL|Invalid staff, patient or medicine tag');
45 }
46
47 if ((int)$item[$i['recallflag']] === 1) {
48     create_alert('critical', 'recall', 'Recalled medicine scanned:
        ' . $item[$i['itemname']], 'index.php?page=medications');
49     createTask('recall-scan-' . $item['id'] . '-' . date('YmdH'),
        'medication', 'Recalled medicine intervention', 'Stop and
        remove ' . $item[$i['itemname']], 0, (int)$item['id'], '
        critical', date('Y-m-d H:i:s', time() + 600));
50     exit('FAIL|Medicine is recalled');
51 }
52
53 if (!empty($item[$i['expirydate']]) && strtotime($item[$i['
    expirydate']]) < time()) {
54     create_alert('critical', 'expiry', 'Expired medicine scanned:
        ' . $item[$i['itemname']], 'index.php?page=medications');
55     createTask('expired-scan-' . $item['id'] . '-' . date('YmdH'),
        'medication', 'Expired medicine intervention', 'Medicine
        expired: ' . $item[$i['itemname']], 0, (int)$item['id'], '
        critical', date('Y-m-d H:i:s', time() + 600));
56     exit('FAIL|Medicine expired');
57 }
58
59 $sched = one(
60     "SELECT *
61     FROM medicationschedule
62     WHERE '{$s['patientid']}'=' ' . (int)$patient['id'] . "
63         AND '{$s['itemid']}'=' ' . (int)$item['id'] . "
64         AND status='pending'
65     ORDER BY '{$s['scheduledtime']}' ASC
66     LIMIT 1"
67 );
68
69 if (!$sched) {
70     create_alert('warning', 'medication', 'Medication verification
        failed for ' . $patient['fullname'], 'index.php?page=
        medications');
71     createTask('medfail-' . $patient['id'] . '-' . $item['id'] . '
        -' . date('YmdHi'), 'medication', 'Unscheduled medication
        scan', $item[$i['itemname']] . ' scanned for ' . $patient['

```

```

        fullname'] . ' without matching pending order', (int)
        $patient['id'], (int)$item['id'], 'high', date('Y-m-d H:i:s
        ', time() + 900));
72     exit('FAIL|No pending medication found');
73 }
74
75 $mins = abs((time() - strtotime($sched[$s['scheduledtime']])) /
        60);
76 $comp = $mins <= 30 ? 'on_time' : 'late';
77
78 $sql = "UPDATE medicationschedule
79     SET status='administered',
80         '{$s['compliancestatus']}'=?,
81         '{$s['verifiedstaffuid']}'=?,
82         '{$s['verifiedpatientuid']}'=?,
83         '{$s['verifiedmedicineuid']}'=?,
84         '{$s['verifiedat']}'=NOW()
85     WHERE id=?";
86 $st = $db->prepare($sql);
87 $st->bind_param('ssssi', $comp, $staffUid, $patientUid, $itemUid,
        $sched['id']);
88 $st->execute();
89
90 $note = 'Verified by RFID via ' . $readerid;
91 $st = $db->prepare("INSERT INTO medicationadministrations(
        scheduleid,patientid,itemid,staffid,notetext) VALUES(?,?,?,?,?)
        ");
92 if ($st) {
93     $st->bind_param('iiiiis', $sched['id'], $patient['id'], $item['
        id'], $staff['id'], $note);
94     $st->execute();
95 }
96
97 $pass = 'pass';
98 $msg = 'Medication administered successfully';
99 $st = $db->prepare("INSERT INTO medicationverifications(scheduleid
        ,patientid,staffid,itemid,resultttext,messagestext) VALUES
        (?,?,?,?,?,?,?)");
100 if (!empty($item['batchno'])) {
101     $bn = $db->real_escape_string($item['batchno']);
102     $db->query("INSERT INTO batch_traces(batchno,item_id,patient_id,
        location_id,scan_time)
103         VALUES('{$bn}'," . (int)$item['id'] . "," . (int)$patient['
        id'] . "," . (int)$reader['id'] . ",NOW())");
104 }
105 if ($st) {
106     $st->bind_param('iiiiss', $sched['id'], $patient['id'], $staff
        ['id'], $item['id'], $pass, $msg);

```

```

107     $st->execute();
108 }
109
110 q("INSERT INTO scanlogs(uid, '{$s1['itemid']}' , '{$s1['patientid']}', '{$s1['staffid']}' , '{$s1['tolocationid']}' , '{$s1['readerid']}' , '{$s1['actiontype']}' , notes)
111     VALUES('" . $db->real_escape_string($itemid) . "', " . (int)
        $itemid . ", " . (int)$patient['id'] . ", " . (int)
        $staff['id'] . ", " . (int)$reader['id'] . ", '" . $db->
        real_escape_string($readerid) . "', 'medverify', 'RFID
        medication administration')");
112
113     updatePatientSeen((int)$patient['id'], (int)$reader['id']);
114     exit('OK|Medication verified and administered');
115 }
116
117 $uid = strtoupper(getv('uid'));
118 if (!$uid) exit('ERROR|Missing uid');
119
120 $item = one("SELECT * FROM items WHERE uid='" . $db->
        real_escape_string($uid) . "' LIMIT 1");
121 if ($item) {
122     $from = (int)($item[$i['locationid']] ?: 0);
123     $to = (int)$reader['id'];
124     $action = $from == $to ? 'scanned' : 'transfer';
125     $newStatus = ((int)$item[$i['recallflag']] == 1) ? 'expired' : '
        instock';
126
127     $sql = "UPDATE items SET '{$i['locationid']}'=?, status=?, '{$i['
        lastseenat']}'?=NOW() WHERE id=?";
128     $st = $db->prepare($sql);
129     $st->bind_param('isi', $to, $newStatus, $item['id']);
130     $st->execute();
131
132     q("INSERT INTO scanlogs(uid, '{$s1['itemid']}' , '{$s1['
        fromlocationid']}' , '{$s1['tolocationid']}' , '{$s1['readerid']
        '}]' , '{$s1['actiontype']}' , notes)
133     VALUES('" . $db->real_escape_string($uid) . "', " . (int)$item[
        'id'] . ", {$from}, {$to}, '" . $db->real_escape_string(
        $readerid) . "', '{$action}', 'Item scan')");
134
135     if (!empty($item[$i['expirydate']]) && strtotime($item[$i['
        expirydate']]) < time()) {
136         create_alert('critical', 'expiry', 'Expired item scanned: ' .
            $item[$i['itemname']], 'index.php?page=inventory');
137         createTask('expired-item-' . $item['id'] . '-' . date('YmdH'),
            'inventory', 'Expired item scanned', $item[$i['itemname']]
            . ' scanned at ' . $reader[$l['locationname']], 0, (int)

```

```

        $item['id'], 'critical', date('Y-m-d H:i:s', time() + 900))
        ;
138     }
139
140     if ((int)$item[$i['recallflag']] === 1) {
141         create_alert('critical', 'recall', 'Recalled item moved: ' .
142             $item[$i['itemname']], 'index.php?page=inventory');
143         createTask('recall-item-' . $item['id'] . '-' . date('YmdH'),
144             'inventory', 'Recalled item moved', $item[$i['itemname']] .
145             ' moved through ' . $reader[$l['locationname']], 0, (int)
146             $item['id'], 'critical', date('Y-m-d H:i:s', time() + 900))
147         ;
148     }
149
150     // Low stock alert
151     $updatedQty = (int)$item['quantity'];
152     $threshold = (int)($item['reorder_threshold'] ?? 10);
153     if ($updatedQty <= $threshold) {
154         create_alert('warning', 'lowstock', 'Low stock: ' . $item[$i['itemname']] .
155             ' (' . $updatedQty . ' left)', 'index.php?page=reorder');
156         createTask('lowstock-' . $item['id'] . '-' . date('Ymd'), 'inventory', '
157             Reorder required: ' . $item[$i['itemname']],
158             'Qty ' . $updatedQty . ' <= threshold ' . $threshold . ' Suggest
159             reorder: ' . ($item['reorder_qty'] ?? 50),
160             0, (int)$item['id'], 'high', date('Y-m-d H:i:s', time() + 3600));
161     }
162
163     // Batch trace
164     if (!empty($item['batchno'])) {
165         $bn = $db->real_escape_string($item['batchno']);
166         $db->query("INSERT INTO batch_traces(batchno,item_id,patient_id,
167             location_id,scan_time)
168             VALUES('{ $bn }','" . (int)$item['id'] . "',0,{ $to },NOW())");
169     }
170
171     exit('OK|' . $item[$i['itemname']] . ' ' . $action);
172 }
173
174 $patient = one("SELECT * FROM patients WHERE '{$p['rfiduid']}'=''" .
175     $db->real_escape_string($uid) . "' LIMIT 1");
176 if ($patient) {
177     updatePatientSeen((int)$patient['id'], (int)$reader['id']);
178
179     q("INSERT INTO scanlogs(uid, '{$sl['patientid']}' , '{$sl['
180         tolocationid']}' , '{$sl['readerid']}' , '{$sl['actiontype']}' ,
181         notes)
182         VALUES('" . $db->real_escape_string($uid) . "', " . (int)
183             $patient['id'] . ", " . (int)$reader['id'] . ", '" . $db->
184             real_escape_string($readerid) . "', 'patientscan', 'Patient

```



```

        seen at reader')");
170
171 $sp = one("SELECT * FROM patientsafetyprofiles WHERE patientid=" .
        (int)$patient['id'] . " LIMIT 1");
172
173 if ($sp) {
174     if (!empty($sp['restrictedlocations']) && csvHasId($sp['
        restrictedlocations'], $reader['id'])) {
175         create_alert('critical', 'safety', $patient['fullname'] .
            ' entered restricted location ' . $reader[$l['
            locationname']], 'index.php?page=safety');
176         createTask('restricted-' . $patient['id'] . '-' . $reader[
            'id'] . '-' . date('YmdHi'), 'safety', 'Restricted area
            breach', $patient['fullname'] . ' entered ' . $reader[
            $l['locationname']], (int)$patient['id'], 0, 'critical'
            , date('Y-m-d H:i:s', time() + 600));
177     }
178
179     if (!empty($sp['allowedlocations']) && !csvHasId($sp['
        allowedlocations'], $reader['id'])) {
180         create_alert('warning', 'safety', $patient['fullname'] . '
            moved outside safe zone to ' . $reader[$l['
            locationname']], 'index.php?page=safety');
181         createTask('safezone-' . $patient['id'] . '-' . $reader['
            id'] . '-' . date('YmdHi'), 'safety', 'Safe-zone
            deviation', $patient['fullname'] . ' moved to ' .
            $reader[$l['locationname']], (int)$patient['id'], 0, '
            high', date('Y-m-d H:i:s', time() + 900));
182     }
183
184     if (($reader[$l['locationtype']] ?? '') === 'washroom') {
185         createTask('washroom-' . $patient['id'] . '-' . date('
            YmdHi'), 'safety', 'Washroom watch', $patient['fullname']
            . ' entered washroom area', (int)$patient['id'], 0,
            'medium', date('Y-m-d H:i:s', time() + ((int)$sp['
            washroomlimitminutes'] * 60)));
186     }
187
188     if (($reader[$l['locationtype']] ?? '') === 'exit') {
189         create_alert('critical', 'safety', $patient['fullname'] .
            ' reached exit reader', 'index.php?page=safety');
190         createTask('exit-' . $patient['id'] . '-' . date('YmdHi'),
            'safety', 'Exit proximity alert', $patient['fullname']
            . ' scanned at exit point', (int)$patient['id'], 0, '
            critical', date('Y-m-d H:i:s', time() + 300));
191     }
192 }
193
    exit('OK|Patient ' . $patient['fullname'] . ' scanned');

```

```

194 }
195
196 $staff = one("SELECT * FROM staffmembers WHERE rfiduid='" . $db->
    real_escape_string($uid) . "' LIMIT 1");
197 if ($staff) {
198     q("INSERT INTO scanlogs(uid, '{$s1['staffid']}' , '{$s1['
        tolocationid']}' , '{$s1['readerid']}' , '{$s1['actiontype']}' ,
        notes)
199     VALUES('" . $db->real_escape_string($uid) . "', " . (int)$staff
        ['id'] . ", " . (int)$reader['id'] . ", '" . $db->
        real_escape_string($readerid) . "', 'staffscan', 'Staff seen
        at reader')");
200     exit('OK|Staff ' . $staff['fullname'] . ' scanned');
201 }
202
203 create_alert('warning', 'unknowntag', 'Unknown RFID tag scanned: ' .
    $uid, 'index.php?page=alerts');
204 createTask('unknown-' . $uid . '-' . date('YmdH'), 'ops', 'Unknown
    RFID tag', 'Unregistered tag ' . $uid . ' scanned at ' . $reader[$l
    ['locationname']], 0, 0, 'medium', date('Y-m-d H:i:s', time() +
    1800));
205 q("INSERT INTO scanlogs(uid, '{$s1['tolocationid']}' , '{$s1['readerid
    ']}', '{$s1['actiontype']}' , notes)
206     VALUES('" . $db->real_escape_string($uid) . "', " . (int)$reader['
        id'] . ", '" . $db->real_escape_string($readerid) . "', 'unknown
        ', 'Unknown tag')");
207 exit('UNKNOWN|Unregistered tag');

```

Listing A.3: scan.php – RFID event intake, medication verification, and rule engine

A.2.2 Reader Heartbeat Endpoint

```

1 <?php
2 require_once 'core.php';
3
4 $l = location_cols();
5 $readerid = getv('readerid');
6 $apikey = getv('apikey');
7
8 if (!$readerid || !$apikey) {
9     exit('ERROR');
10 }
11
12 $sql = "UPDATE locations
13     SET '{$l['lastheartbeat']}'=NOW()
14     WHERE '{$l['readerid']}'=? AND '{$l['apikey']}'=? AND '{$l['
        isactive']}'=1";

```

```

15 $st = $db->prepare($sql);
16 $st->bind_param('ss', $readerid, $apikey);
17 $st->execute();
18
19 echo $st->affected_rows > 0 ? 'OK' : 'ERROR';

```

Listing A.4: heartbeat.php – reader liveness update

A.2.3 Offline Scan Queue Reconciliation Endpoint

```

1 <?php
2 require_once 'core.php';
3 header('Content-Type: application/json');
4
5 $l = location_cols();
6 $readerid = $_POST['readerid'] ?? getv('readerid');
7 $apikey = $_POST['apikey'] ?? getv('apikey');
8
9 if (!$readerid || !$apikey) exit(json_encode(['ok'=>false, 'msg'=>'
    Missing credentials']));
10
11 $reader = one("SELECT * FROM locations
12               WHERE '{$l['readerid']}'='\".$db->real_escape_string(
13                   $readerid).\"'
14               AND '{$l['apikey']}'='\".$db->real_escape_string($apikey
15                   ).\"'
16               AND '{$l['isactive']}'=1 LIMIT 1");
17 if (!$reader) { http_response_code(403); exit(json_encode(['ok'=>false
18     , 'msg'=>'Unauthorized'])); }
19
20 $body = json_decode(file_get_contents('php://input'), true);
21 if (!is_array($body)) exit(json_encode(['ok'=>false, 'msg'=>'Invalid
22     JSON']));
23
24 $n = 0;
25 foreach ($body as $scan) {
26     $uid = strtoupper($db->real_escape_string($scan['uid'] ?? ''));
27     $qat = $db->real_escape_string($scan['queued_at'] ?? date('Y-m-d H
28         :i:s'));
29     if (!$uid) continue;
30     $db->query("INSERT INTO offline_scan_queue(readerid,uid,queued_at)
31               VALUES('\".$db->real_escape_string($readerid).\"', '{ $uid
32                   }', '{ $qat }')");
33     $n++;
34 }
35 echo json_encode(['ok'=>true, 'queued'=>$n]);

```

Listing A.5: offline_queue.php – bulk reconciliation of buffered scans after a network outage

A.2.4 Live Dashboard Data Endpoint

```

1  <?php
2  require_once 'core.php';
3  require_login();
4  header('Content-Type: application/json');
5
6  $s1 = scanlog_cols();
7  $i  = item_cols();
8  $l  = location_cols();
9  $a  = alert_cols();
10
11 $k = one(
12     "SELECT
13         (SELECT COUNT(*) FROM patients WHERE status<>'discharged') AS
           patients,
14         (SELECT COUNT(*) FROM items) AS items,
15         (SELECT COUNT(*) FROM alerts WHERE '{$a['isresolved']}'=0) AS
           alerts,
16         (SELECT COUNT(*) FROM workflowtasks WHERE status<>'done') AS
           tasks,
17         (SELECT COUNT(*) FROM scanlogs WHERE DATE('{$s1['scantime']}')
           =CURDATE()) AS scans,
18         (SELECT COUNT(*) FROM locations WHERE '{$l['readerid']}' IS
           NOT NULL
19         AND ('{$l['lastheartbeat']}' IS NULL OR '{$l['
           lastheartbeat']}' < NOW() - INTERVAL 10 MINUTE)) AS
           offline"
20 );
21
22 $recentMoves = all_rows(
23     "SELECT
24         s.id,
25         s.uid,
26         s. '{$s1['actiontype']}' AS action_type,
27         s. '{$s1['scantime']}' AS scan_time,
28         p.fullname AS patient_name,
29         i. '{$i['itemname']}' AS item_name,
30         l1. '{$l['locationname']}' AS from_name,
31         l2. '{$l['locationname']}' AS to_name
32     FROM scanlogs s
33     LEFT JOIN patients p ON p.id = s. '{$s1['patientid']}'

```

```

34     LEFT JOIN items i ON i.id = s.{'$s1['itemid']}'
35     LEFT JOIN locations l1 ON l1.id = s.{'$s1['fromlocationid']}'
36     LEFT JOIN locations l2 ON l2.id = s.{'$s1['tolocationid']}'
37     ORDER BY s.id DESC
38     LIMIT 10"
39 );
40
41 $alerts = all_rows(
42     "SELECT id, severity, {'$a['type']}' AS alert_type, message, {'$a
43         ['createdat']}' AS created_at, {'$a['isresolved']}' AS
44         is_resolved
45     FROM alerts
46     ORDER BY {'$a['isresolved']}' , id DESC
47     LIMIT 10"
48 );
49
50 echo json_encode([
51     'ok'          => true,
52     'kpi'         => $k,
53     'moves'       => $recentMoves,
54     'alerts'      => $alerts,
55     'server_time' => date('Y-m-d H:i:s')
56 ], JSON_UNESCAPED_UNICODE);

```

Listing A.6: live_updates.php – JSON KPI, recent-activity, and alert feed for the dashboard

A.3 Caretaker-Facing Endpoints

A.3.1 Persistent Caretaker Login

```

1 <?php
2 require_once 'core.php';
3 if (session_status()===PHP_SESSION_NONE) session_start();
4 $msg = '';
5 if ($_SERVER['REQUEST_METHOD']==='POST') {
6     $u = posted('username');
7     $p = posted('password');
8     $st = $db->prepare("SELECT ca.*, p.fullname AS patient_name, p.id
9         AS pid
10             FROM caretaker_accounts ca
11             JOIN patients p ON p.id=ca.patient_id
12             WHERE ca.username=? AND ca.is_active=1 LIMIT 1
13             ");
14     $st->bind_param('s',$u); $st->execute();
15     $row = $st->get_result()->fetch_assoc();

```

```

14     if ($row && password_verify($p, $row['password'])) {
15         $_SESSION['ct_account_id'] = $row['id'];
16         $_SESSION['ct_patient_id'] = $row['pid'];
17         $_SESSION['ct_patient_name'] = $row['patient_name'];
18         header('Location: caretaker_home.php'); exit;
19     }
20     $msg = 'Invalid credentials';
21 }
22 ?><!DOCTYPE html><html><head><meta charset="utf-8">
23 <meta name="viewport" content="width=device-width,initial-scale=1">
24 <title>MediTrack -- Caretaker Login</title>
25 <style>
26 *{box-sizing:border-box;margin:0;padding:0}
27 body{background:#050c14;color:#edf3ff;font-family:Arial,sans-serif;
28     display:flex;align-items:center;justify-content:center;min-height
29     :100vh;padding:20px}
30 .card{background:#0b1017;border:1px solid #1a2636;border-radius:18px;
31     padding:36px;width:100%;max-width:400px}
32 h2{margin-bottom:6px;font-size:22px;color:#56a8ff}
33 .sub{color:#90a4c2;font-size:13px;margin-bottom:24px}
34 label{display:block;font-size:13px;color:#90a4c2;margin-bottom:5px}
35 input{width:100%;background:#101722;border:1px solid #1a2636;border-
36     radius:10px;padding:12px;color:#edf3ff;font-size:14px;margin-bottom
37     :16px}
38 button{width:100%;background:#56a8ff;color:#000;border:none;border-
39     radius:10px;padding:13px;font-size:15px;font-weight:700;cursor:
40     pointer}
41 .err{background:rgba(251,113,133,.1);border:1px solid #fb7185;color:#
42     fb7185;border-radius:10px;padding:12px;margin-bottom:16px;font-size
43     :13px}
44 </style></head><body>
45 <div class="card">
46     <h2>MediTrack</h2>
47     <div class="sub">Caretaker Portal -- Patient Updates</div>
48     <?php if ($msg): ?><div class="err"><?= e($msg) ?></div><?php
49         endif; ?>
50     <form method="post">
51         <label>Username</label><input name="username" required
52             autocomplete="username">
53         <label>Password</label><input type="password" name="password"
54             required autocomplete="current-password">
55         <button type="submit">Login</button>
56     </form>
57 </div>
58 </body></html>

```

Listing A.7: caretaker_login.php – persistent caretaker account authentication

A.3.2 Expiring Tokenised Caretaker Portal

```

1 <?php
2 require_once 'core.php';
3
4 $tok = token_cols();
5 $v = vitals_cols();
6 $s = schedule_cols();
7 $i = item_cols();
8 $ivc = iv_cols();
9
10 $token = getv('token');
11 if (!$token) die('Missing token');
12
13 $row = one(
14     "SELECT
15         ct.*,
16         c.fullname AS caretaker_name,
17         p.fullname AS patient_name
18     FROM caretaker_tokens ct
19     JOIN caretakers c ON c.id = ct.{'$tok['caretakerid']}'
20     JOIN patients p ON p.id = ct.{'$tok['patientid']}'
21     WHERE ct.token='\" . $db->real_escape_string($token) . '\"
22           AND ct.{'$tok['isactive']}'=1
23           AND ct.{'$tok['expiresat']}' > NOW()
24     LIMIT 1"
25 );
26 if (!$row) die('Invalid or expired token');
27
28 $vitals = all_rows(
29     "SELECT *
30     FROM patientvitals
31     WHERE {'$v['patientid']}'='\" . (int)$row[$tok['patientid']] . "\"
32     ORDER BY {'$v['recordedat']}' DESC
33     LIMIT 5"
34 );
35
36 $meds = all_rows(
37     "SELECT
38         ms.*,
39         i.{'$i['itemname']}' AS item_name
40     FROM medicationschedule ms
41     JOIN items i ON i.id=ms.{'$s['itemid']}'
42     WHERE ms.{'$s['patientid']}'='\" . (int)$row[$tok['patientid']] . "\"
43     ORDER BY ms.{'$s['scheduledtime']}' DESC
44     LIMIT 10"
45 );
46

```

```

47 $ivs = all_rows(
48     "SELECT *
49     FROM iv_drips
50     WHERE '{$ivc['patientid']}'=' ' . (int)$row[$tok['patientid']] . "
51     ORDER BY id DESC
52     LIMIT 5"
53 );
54
55 render_header('Caretaker Portal', 'portal', false);
56 ?>
57 <div style="max-width:1100px;margin:0 auto">
58     <div class="card">
59         <h3><i class="fas fa-user-group"></i> Caretaker Access</h3>
60         <div class="muted">Caretaker: <?= e($row['caretaker_name'])
61             ?></div>
62         <div style="font-size:22px;font-weight:800;margin-top:10px"
63             ><?= e($row['patient_name']) ?></div>
64         <div class="muted" style="margin-top:6px">Token valid till <?=
65             e($row[$tok['expiresat']]) ?></div>
66     </div>
67
68     <div class="grid g2" style="margin-top:18px">
69         <div class="card">
70             <h3>Recent Vitals</h3>
71             <table class="table">
72                 <tr><th>Time</th><th>Temp</th><th>BP</th><th>Pulse</th>
73                 <th>SpO2</th></tr>
74                 <?php foreach ($vitals as $x): ?>
75                 <tr>
76                     <td><?= e($x[$v['recordedat']]) ?></td>
77                     <td><?= e($x['temperature']) ?></td>
78                     <td><?= e($x['systolic_bp']) ?>/<?= e($x['
79                         diastolic_bp']) ?></td>
80                     <td><?= e($x['pulse_rate']) ?></td>
81                     <td><?= e($x['spo2']) ?></td>
82                 </tr>
83                 <?php endforeach; ?>
84             </table>
85         </div>
86
87         <div class="card">
88             <h3>Medication Updates</h3>
89             <table class="table">
90                 <tr><th>Time</th><th>Medicine</th><th>Status</th><th>
91                     Compliance</th></tr>
92                 <?php foreach ($meds as $x): ?>
93                 <tr>
94                     <td><?= e($x[$s['scheduledtime']]) ?></td>

```



```

89         <td><?= e($x['item_name']) ?></td>
90         <td><?= e($x['status']) ?></td>
91         <td><?= e($x[$s['compliancestatus']] ?? ' ') ?></td>
92     >
93     </tr>
94     <?php endforeach; ?>
95 </table>
96 </div>
97
98 <div class="card" style="margin-top:18px">
99     <h3>IV Drips</h3>
100     <table class="table">
101         <tr><th>Fluid</th><th>Remaining</th><th>Rate</th><th>ETA</th><th>Status</th></tr>
102         <?php foreach ($ivs as $x): ?>
103         <tr>
104             <td><?= e($x[$ivc['fluidname']]) ?></td>
105             <td><?= e($x[$ivc['remainingml']]) ?> ml</td>
106             <td><?= e($x[$ivc['flowratemlhr']]) ?> ml/hr</td>
107             <td><?= e($x[$ivc['etaend']]) ?></td>
108             <td><?= e($x['status']) ?></td>
109         </tr>
110         <?php endforeach; ?>
111     </table>
112 </div>
113 </div>
114 <?php render_footer(false); ?>

```

Listing A.8: caretaker_portal.php – token-validated, read-only caretaker view

A.4 Reporting Endpoint

```

1 <?php
2 require_once 'core.php';
3 require_login();
4
5 $type = getv('type', 'summary');
6
7 $p = patient_cols();
8 $i = item_cols();
9 $s = schedule_cols();
10 $v = vitals_cols();
11 $t = task_cols();
12 $a = alert_cols();
13
14 render_header('Report', 'reports');

```

```

15
16 if ($type === 'patient') {
17     $patientId = (int)getv('patient_id', 0);
18
19     $patient = one("SELECT * FROM patients WHERE id={$patientId} LIMIT
20         1");
21     if (!$patient) {
22         echo '<div class="banner">Patient not found</div>';
23         render_footer();
24         exit;
25     }
26
27     $vitals = all_rows("SELECT * FROM patientvitals WHERE '{$v['
28         patientid']}'= {$patientId} ORDER BY '{$v['recordedat']}' DESC
29         LIMIT 20");
30
31     $tasks = all_rows("SELECT * FROM workflowtasks WHERE '{$t['
32         patientid']}'= {$patientId} ORDER BY '{$t['createdat']}' DESC
33         LIMIT 20");
34
35     $meds = all_rows(
36         "SELECT ms.*, i. '{$i['itemname']}' AS item_name
37         FROM medicationschedule ms
38         JOIN items i ON i.id=ms. '{$s['itemid']}'
39         WHERE ms. '{$s['patientid']}'= {$patientId}
40         ORDER BY ms. '{$s['scheduledtime']}' DESC
41         LIMIT 20"
42     );
43
44     echo '<div class="card">';
45     echo '<h3>Patient Report</h3>';
46     echo '<div style="font-size:22px;font-weight:800"> . e($patient[
47         fullname']) . '</div>';
48     echo '<div class="muted" style="margin-top:6px">Code: ' . e(
49         $patient[$p['patientcode']] ?? '') . '</div>';
50     echo '</div>';
51
52     echo '<div class="grid g2" style="margin-top:18px">';
53
54     echo '<div class="card"><h3>Vitals</h3><table class="table"><tr><
55         th>Time</th><th>Temp</th><th>BP</th><th>Pulse</th><th>SpO2</th>
56         </tr>';
57     foreach ($vitals as $x) {
58         echo '<tr>';
59         echo '<td>' . e($x[$v['recordedat']]) . '</td>';
60         echo '<td>' . e($x['temperature']) . '</td>';
61         echo '<td>' . e($x['systolic_bp']) . '/' . e($x['diastolic_bp']
62             ) . '</td>';
63         echo '<td>' . e($x['pulse_rate']) . '</td>';
64         echo '<td>' . e($x['spo2']) . '</td>';

```

```

53     echo '</tr>';
54 }
55 echo '</table></div>';
56
57 echo '<div class="card"><h3>Workflow Tasks</h3><table class="table
    "><tr><th>Time</th><th>Type</th><th>Title</th><th>Status</th><
    th>Priority</th></tr>';
58 foreach ($tasks as $x) {
59     echo '<tr>';
60     echo '<td>' . e($x[$t['createdat']]) . '</td>';
61     echo '<td>' . e($x[$t['tasktype']]) . '</td>';
62     echo '<td>' . e($x['title']) . '</td>';
63     echo '<td>' . e($x['status']) . '</td>';
64     echo '<td>' . e($x['priority']) . '</td>';
65     echo '</tr>';
66 }
67 echo '</table></div>';
68
69 echo '</div>';
70
71 echo '<div class="card" style="margin-top:18px"><h3>Medication
    History</h3><table class="table"><tr><th>Time</th><th>Medicine
    </th><th>Status</th><th>Compliance</th></tr>';
72 foreach ($meds as $x) {
73     echo '<tr>';
74     echo '<td>' . e($x[$s['scheduledtime']]) . '</td>';
75     echo '<td>' . e($x['item_name']) . '</td>';
76     echo '<td>' . e($x['status']) . '</td>';
77     echo '<td>' . e($x[$s['compliancestatus']] ?? '') . '</td>';
78     echo '</tr>';
79 }
80 echo '</table></div>';
81
82 render_footer();
83 exit;
84 }
85
86 $k = one(
87     "SELECT
88         (SELECT COUNT(*) FROM patients WHERE status<>'discharged') AS
            patients,
89         (SELECT COUNT(*) FROM items) AS items,
90         (SELECT COUNT(*) FROM alerts WHERE '{$a['isresolved']}'=0) AS
            alerts,
91         (SELECT COUNT(*) FROM workflowtasks WHERE status<>'done') AS
            tasks"
92 );
93

```

```

94 $recentAlerts = all_rows(
95     "SELECT id, severity, message, '{$a['createdat']}' AS created_at
96     FROM alerts
97     ORDER BY '{$a['createdat']}' DESC
98     LIMIT 20"
99 );
100
101 ?>
102 <div class="grid g4">
103     <div class="card kpi"><div class="label">Patients</div><div class="
104         "value"><?= e($k['patients']) ?></div></div>
105     <div class="card kpi"><div class="label">Items</div><div class="
106         value"><?= e($k['items']) ?></div></div>
107     <div class="card kpi"><div class="label">Open Alerts</div><div
108         class="value"><?= e($k['alerts']) ?></div></div>
109     <div class="card kpi"><div class="label">Open Tasks</div><div
110         class="value"><?= e($k['tasks']) ?></div></div>
111 </div>
112
113 <div class="card" style="margin-top:18px">
114     <h3>Recent Alerts</h3>
115     <table class="table">
116         <tr><th>Time</th><th>Severity</th><th>Message</th></tr>
117         <?php foreach ($recentAlerts as $x): ?>
118             <tr>
119                 <td><?= e($x['created_at']) ?></td>
120                 <td><?= e($x['severity']) ?></td>
121                 <td><?= e($x['message']) ?></td>
122             </tr>
123         <?php endforeach; ?>
124     </table>
125 </div>
126 <?php render_footer(); ?>

```

Listing A.9: report.php – summary and per-patient report rendering

Database Structure and Abbreviations

B.1 Core Database Tables

The deployed schema, `meditrack_compact`, is normalised to third normal form. Table [B.1](#) summarises the core tables actually referenced by the backend endpoints in Appendix A, including the extension tables added to support batch traceability, offline reconciliation, and dual-mode caretaker access.

Table B.1: MediTrack Database Schema Summary

Table	Purpose
patients	Patient demographics, RFID UID, ward/bed, status
staffmembers	Staff records, RFID UID, active flag
items	Medicine/equipment/asset catalogue, UID, batch, recall/expiry
locations	Reader registry: reader ID, API key, location name/type, heartbeat
scanlogs	Immutable log of every RFID scan event
medicationschedule	Scheduled doses, status, compliance, verification fields
medicationadministration	Confirmed administration records linked to schedule entries
medicationverifications	Pass/fail verification attempts with result messages
batch_traces	Batch number movement history across items/-patients/locations
patientsafetyprofiles	Restricted/allowed location lists, washroom time limit
alerts	Active and resolved safety/inventory alerts
workflowtasks	Automated tasks with status and priority tracking
iv_drips	IV monitoring records per patient
offline_scan_queue	Buffered scans reconciled after a network outage
admins	Administrator login credentials
caretaker_accounts	Persistent caretaker login credentials linked to a patient
caretakers / caretaker_tokens	Expiring tokenised caretaker access records
patientvitals	Recorded vital-sign readings per patient

B.2 Abbreviations

Table B.2: Technical Abbreviations Used in This Report

Abbreviation	Full Form
RFID	Radio Frequency Identification
IoT	Internet of Things
ESP32	Espressif Systems 32-bit Wi-Fi/Bluetooth SoC
UID	Unique Identifier
SPI	Serial Peripheral Interface
HTTP	Hypertext Transfer Protocol
AJAX	Asynchronous JavaScript and XML
SQL	Structured Query Language
API	Application Programming Interface
ICU	Intensive Care Unit
XAMPP	Cross-platform Apache, MariaDB, PHP, Perl stack
AI	Artificial Intelligence
ML	Machine Learning
SDG	Sustainable Development Goal
KPI	Key Performance Indicator
CSRF	Cross-Site Request Forgery
UHF	Ultra High Frequency